

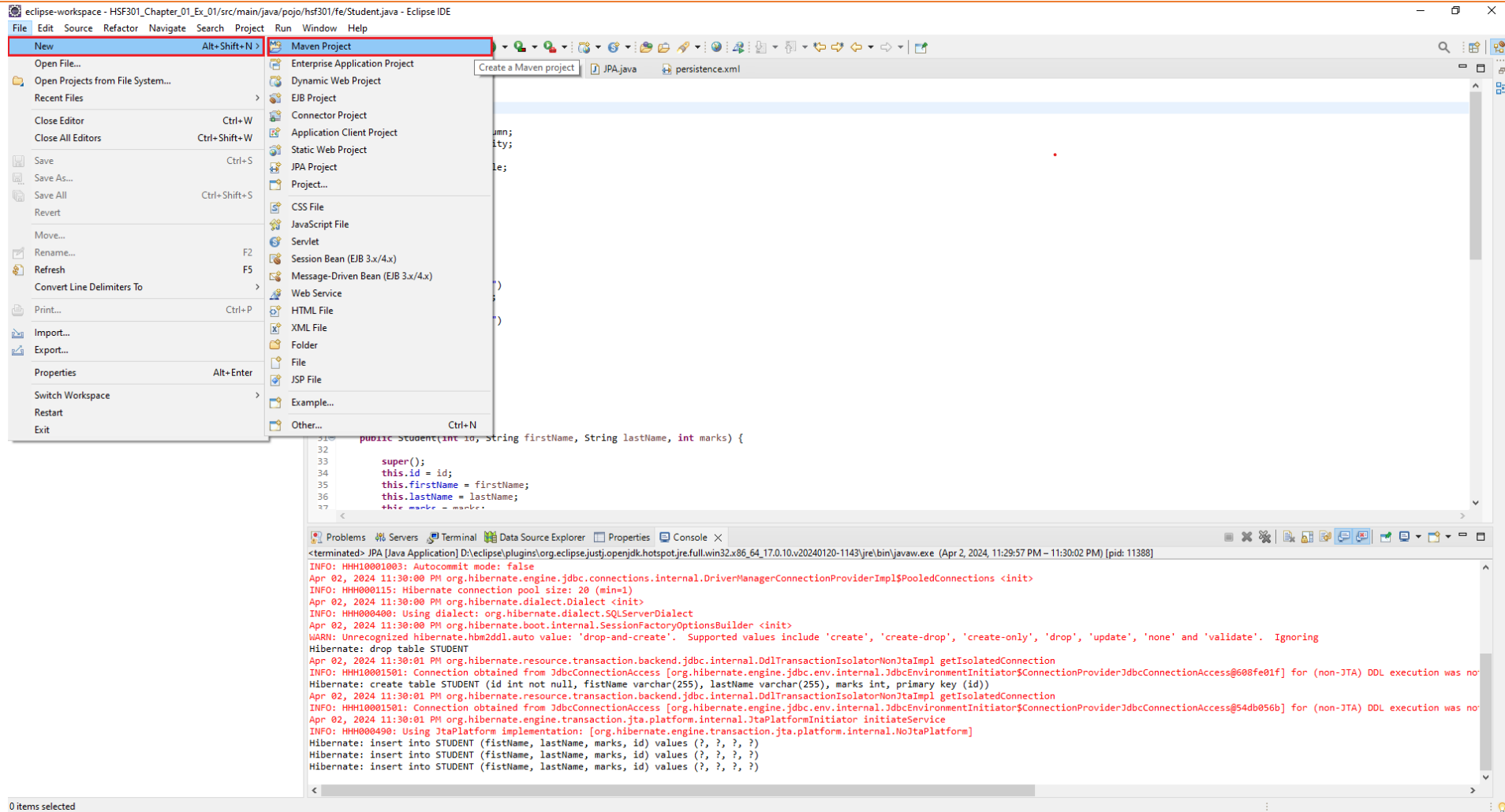
Build a simple application with Data Access using Spring & Spring JDBC, Spring ORM

Objectives

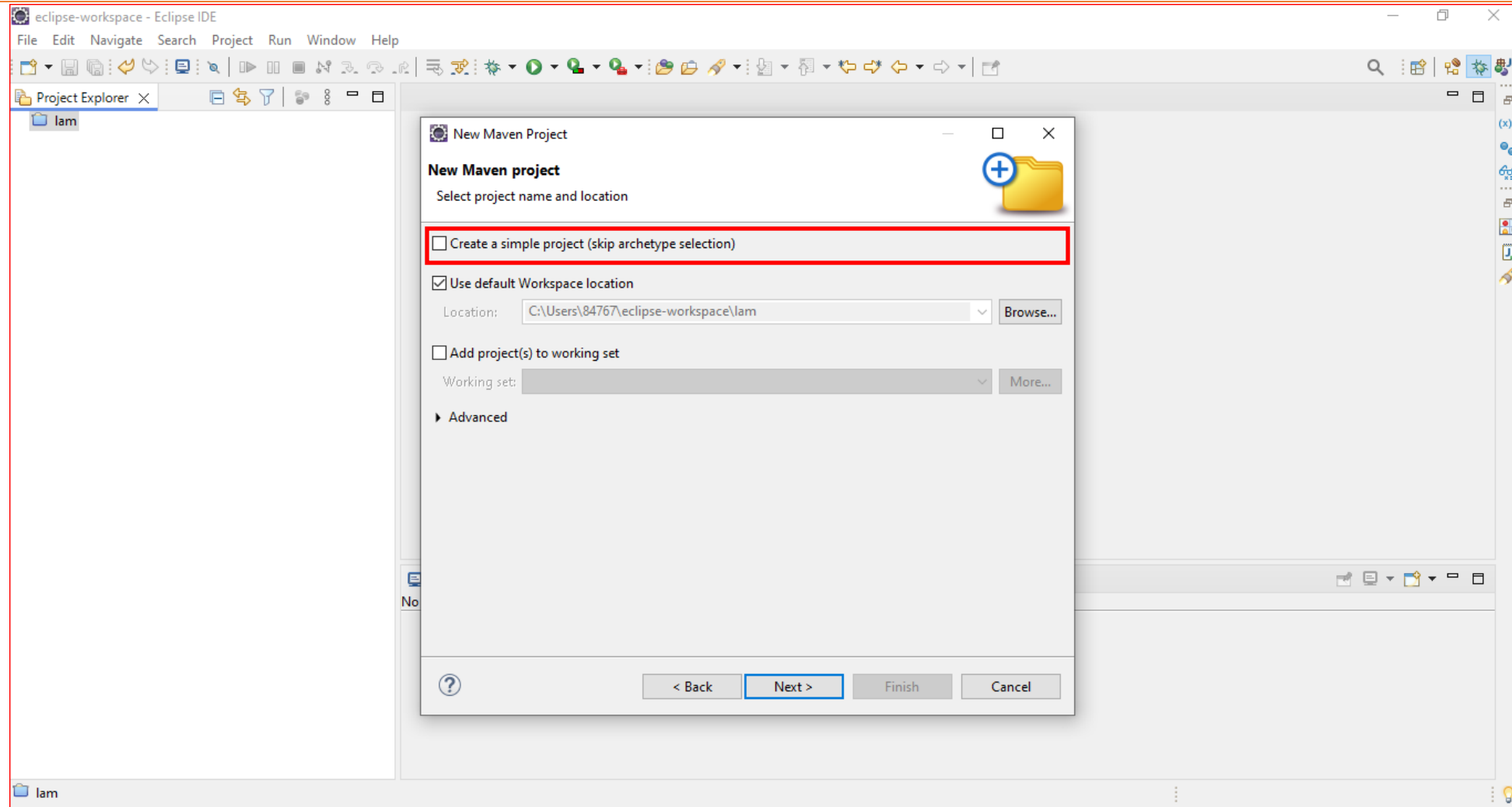
Build Desktop application with Data access

- ◆ Create a new Maven project in Eclipse IDE
- ◆ Add the necessary dependencies for Spring MVC and data access
- ◆ Create the Model - Define the entity classes that represent domain objects
- ◆ Define the data access object (DAO) interfaces and their implementations for interacting with the database.
- ◆ Create the Controller
- ◆ Set Up the Views
- ◆ Create the views using JSP, Thymeleaf, or another templating engine
- ◆ Implement the Business Logic
- ◆ Testing
- ◆ Run the Application

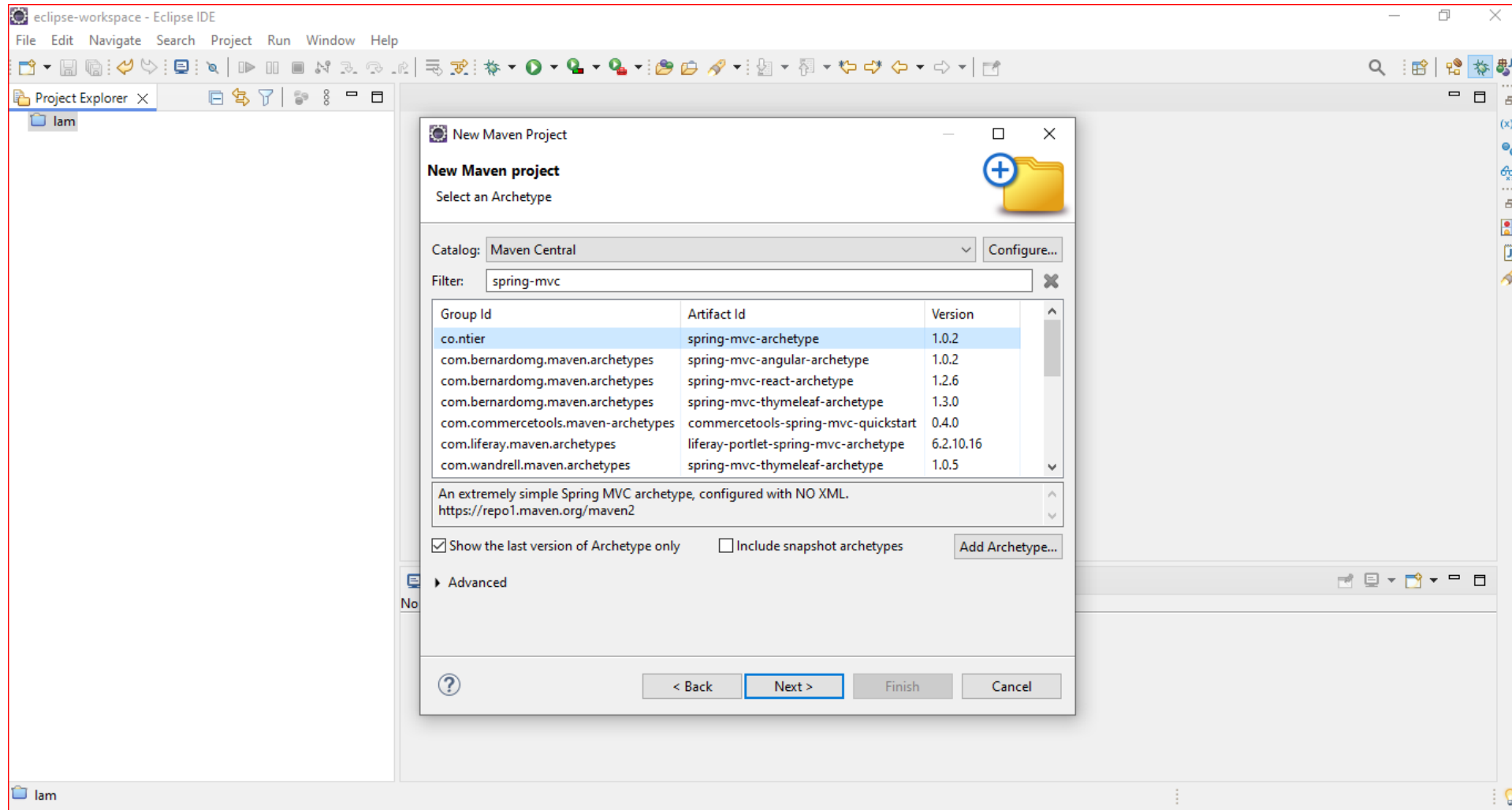
1. Open Eclipse, File | New | Maven Project



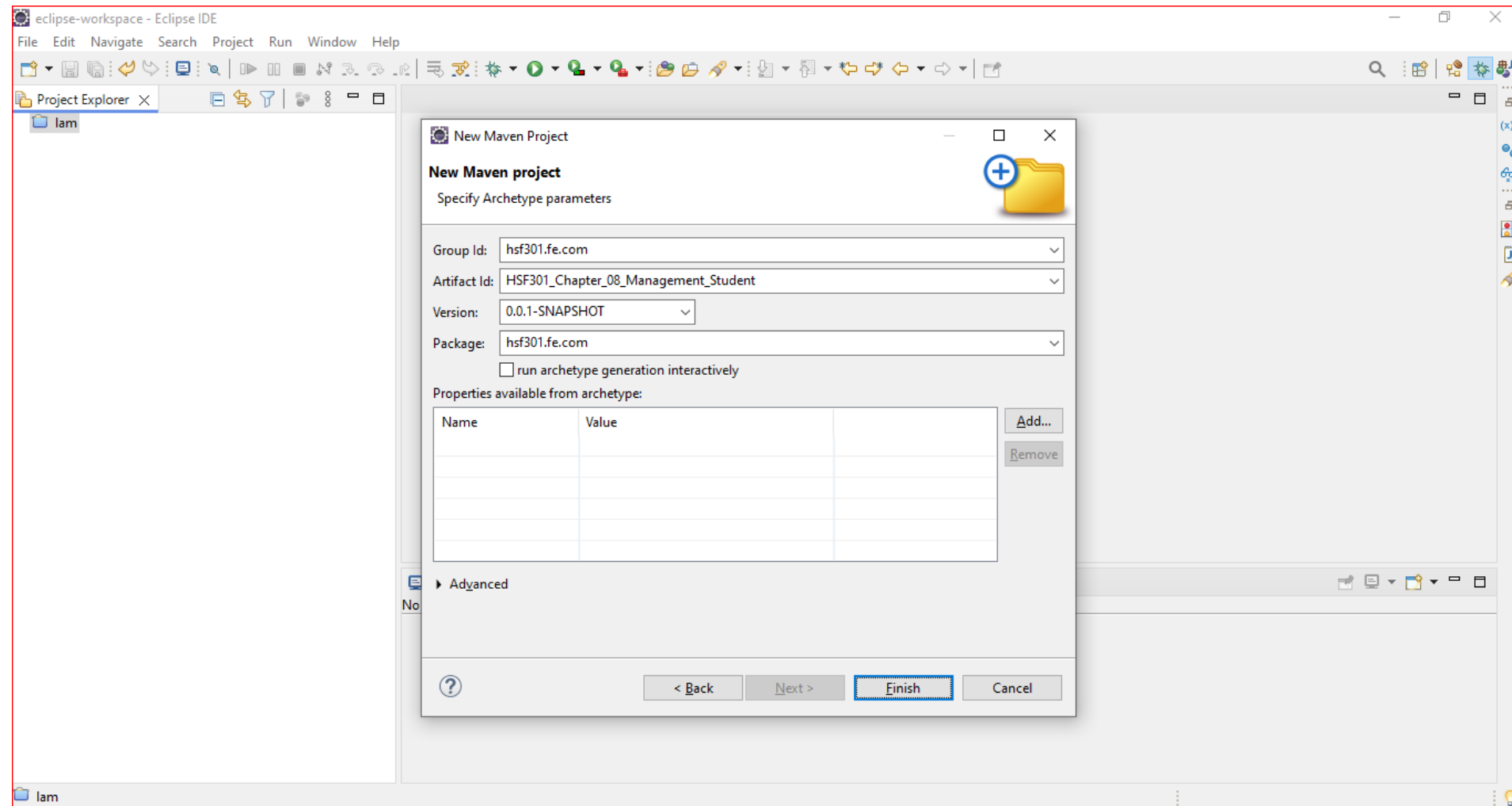
2. Check Create a simple project -> Browse Project -> Next



3. Fill the information Project -> Click Finish



4. Fill the information Project -> Click Finish



5. Structure of Maven Project

The screenshot displays the Eclipse IDE interface for a Maven project named 'HSF301_Chapter_08_Management_Student'. The Project Explorer on the left shows the following structure:

- HSF301_Chapter_08_Management_Student [HSF301 main]
 - Deployment Descriptor: HSF301_Chapter_08_Management_Student
 - Java Resources
 - src/main/java
 - Deployment Descriptor: HSF301_Chapter_08_Management_Student
 - hsf301.fe.com.config
 - Deployment Descriptor: HSF301_Chapter_08_Management_Student
 - MvcConfiguration.java
 - hsf301.fe.com.controller
 - Deployment Descriptor: HSF301_Chapter_08_Management_Student
 - HomeController.java
 - Libraries
 - Deployed Resources
 - src
 - main
 - java
 - webapp
 - resources
 - style.css
 - WEB-INF
 - views
 - home.jsp
 - web.xml
 - target
 - pom.xml

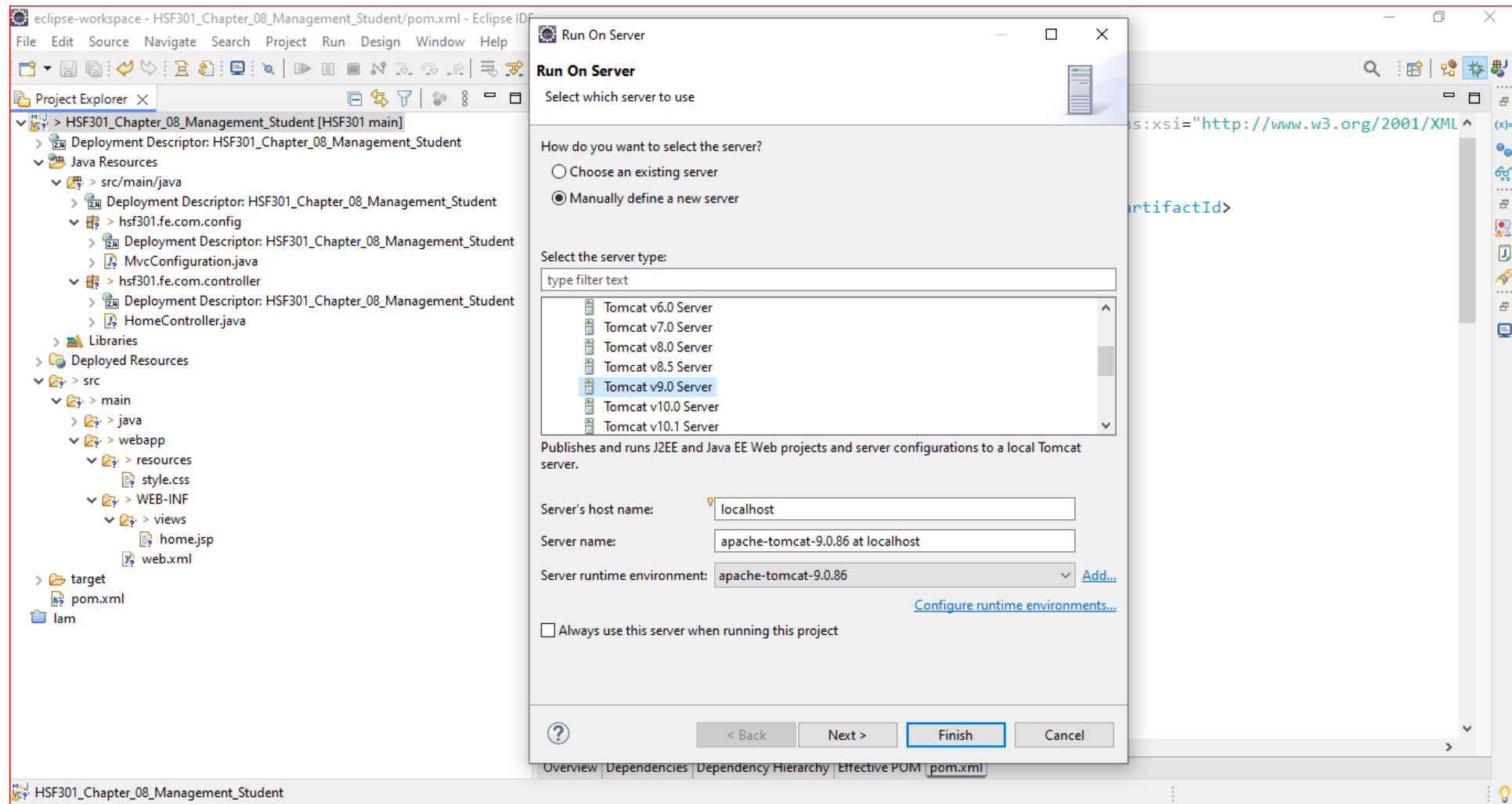
The main editor shows the 'pom.xml' file with the following XML content:

```

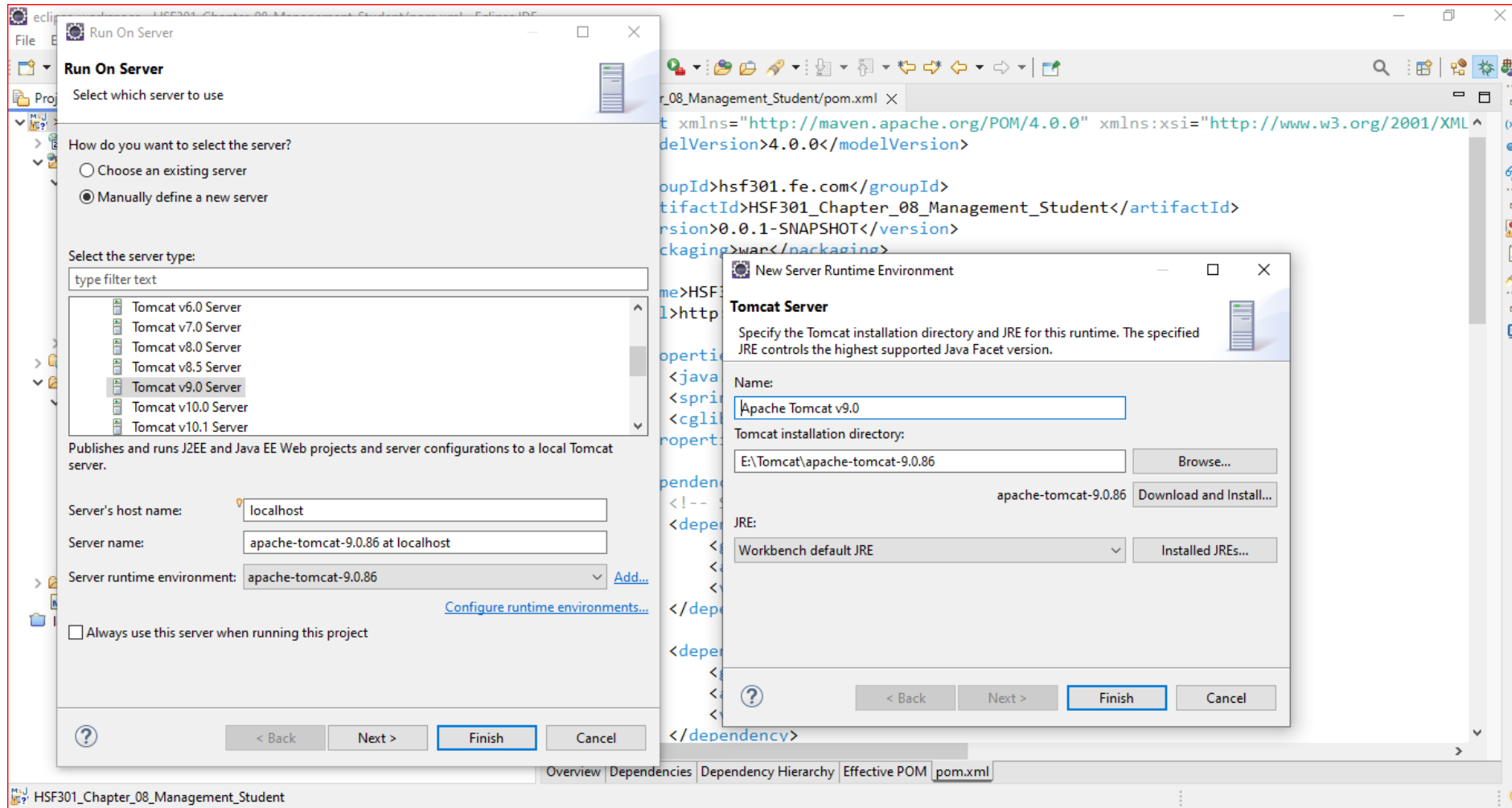
20 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
21   <modelVersion>4.0.0</modelVersion>
22
23   <groupId>hsf301.fe.com</groupId>
24   <artifactId>HSF301_Chapter_08_Management_Student</artifactId>
25   <version>0.0.1-SNAPSHOT</version>
26   <packaging>war</packaging>
27
28   <name>HSF301_Chapter_08_Management_Student</name>
29   <url>http://maven.apache.org</url>
30
31   <properties>
32     <java.version>1.6</java.version>
33     <spring.version>3.1.0.RELEASE</spring.version>
34     <cglib.version>2.2.2</cglib.version>
35   </properties>
36
37   <dependencies>
38     <!-- Spring core & mvc -->
39     <dependency>
40       <groupId>org.springframework</groupId>
41       <artifactId>spring-context</artifactId>
42       <version>${spring.version}</version>
43     </dependency>
44
45     <dependency>
46       <groupId>org.springframework</groupId>
47       <artifactId>spring-webmvc</artifactId>
48       <version>${spring.version}</version>
49     </dependency>
50   </dependencies>
51 </project>
  
```

The bottom status bar shows the 'pom.xml' file is selected, and the 'Overview' tab is active.

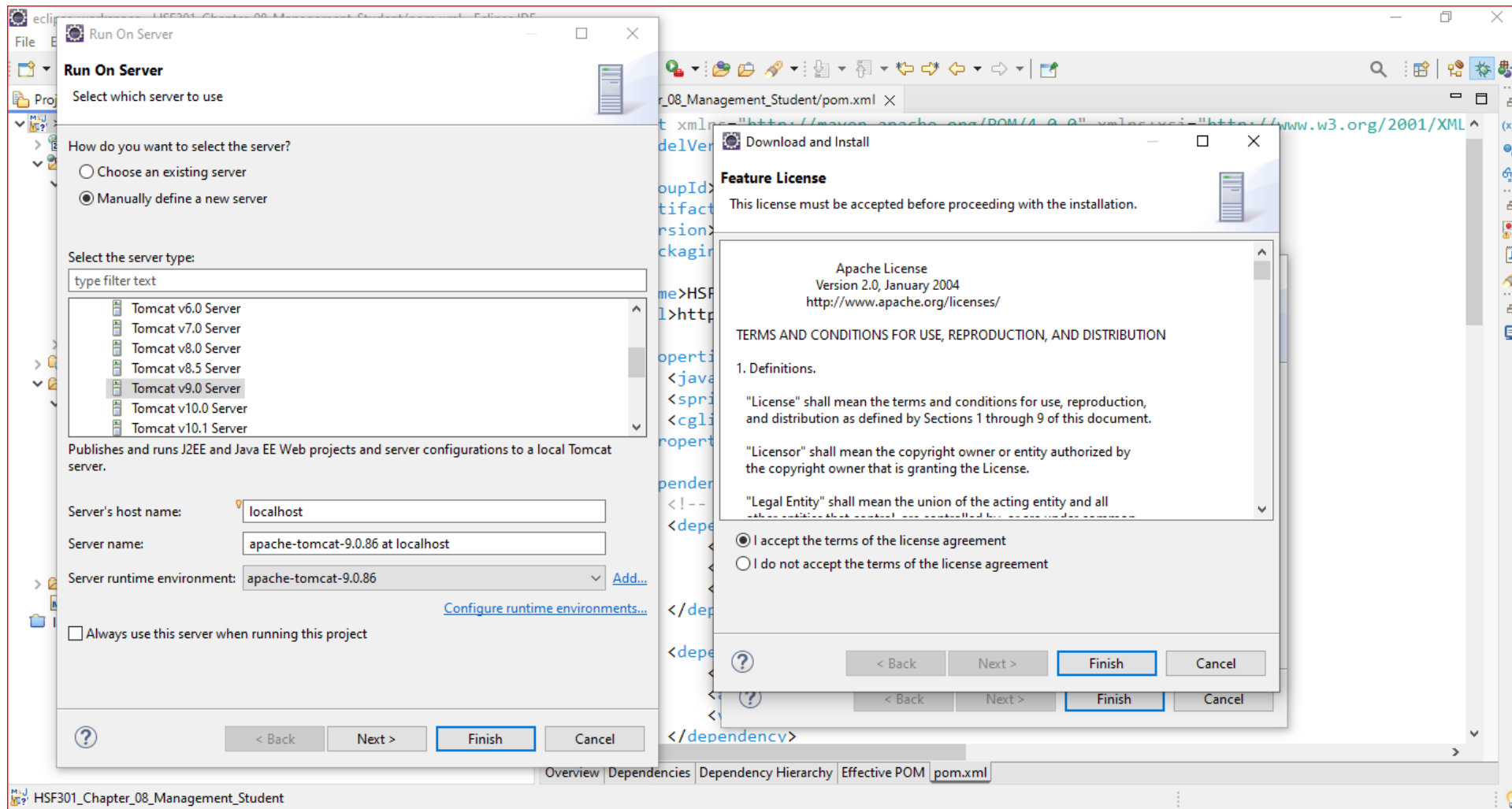
6. Right Click -> Run On Server



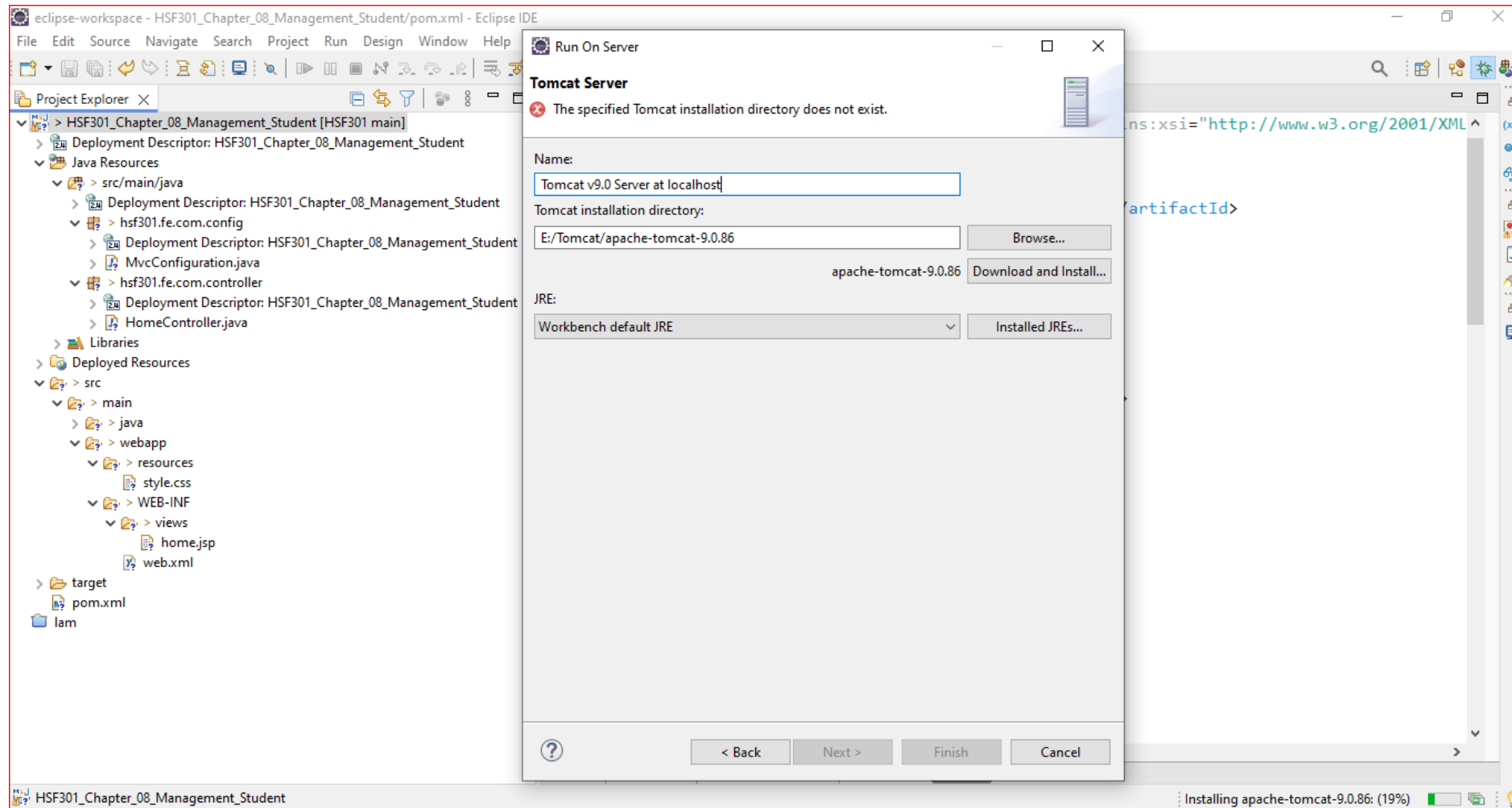
7. Browse the Server Location



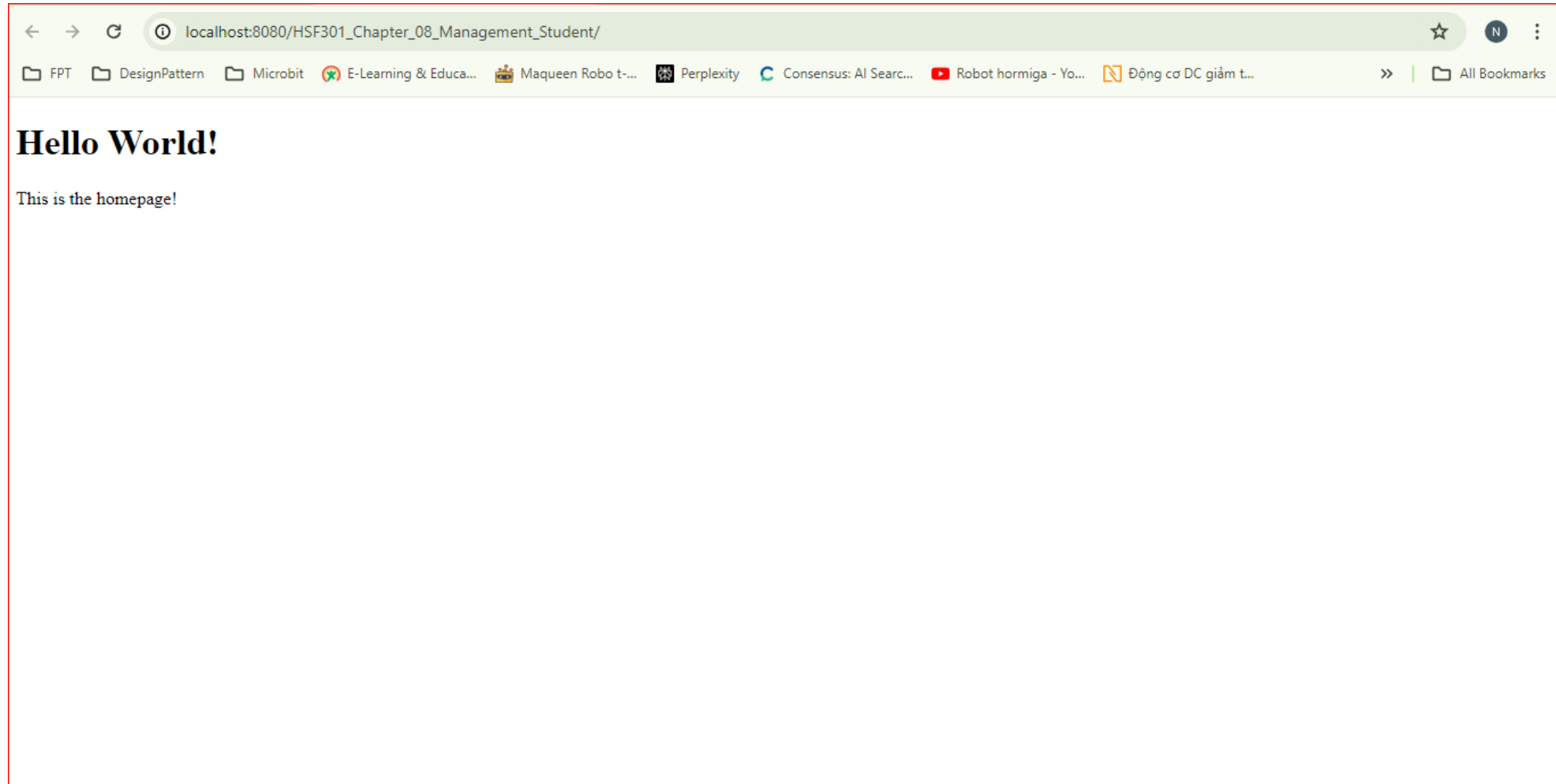
8. Click Download and Install



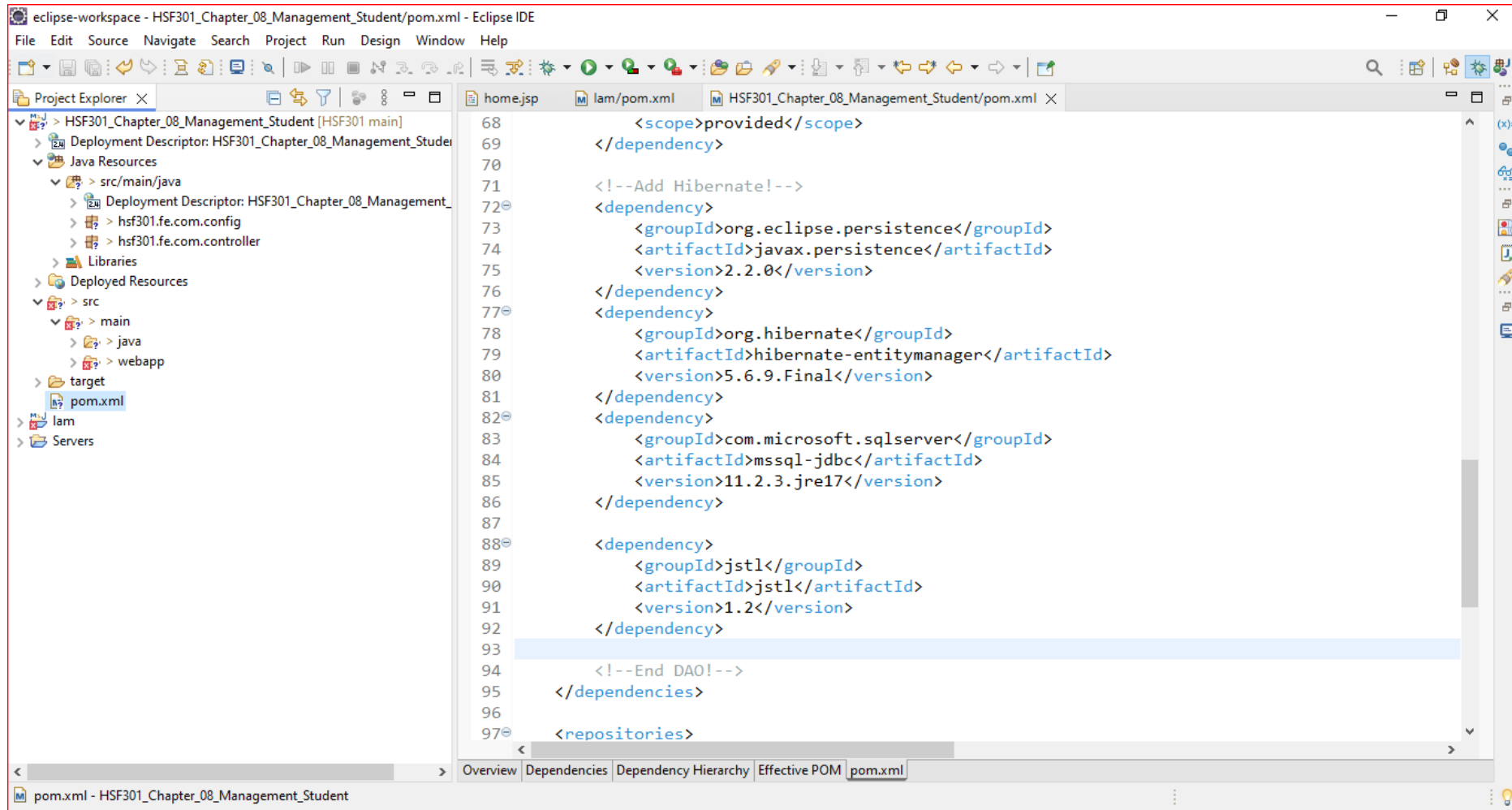
9. Next -> Finish



10. Result



11. Edit pom.xml

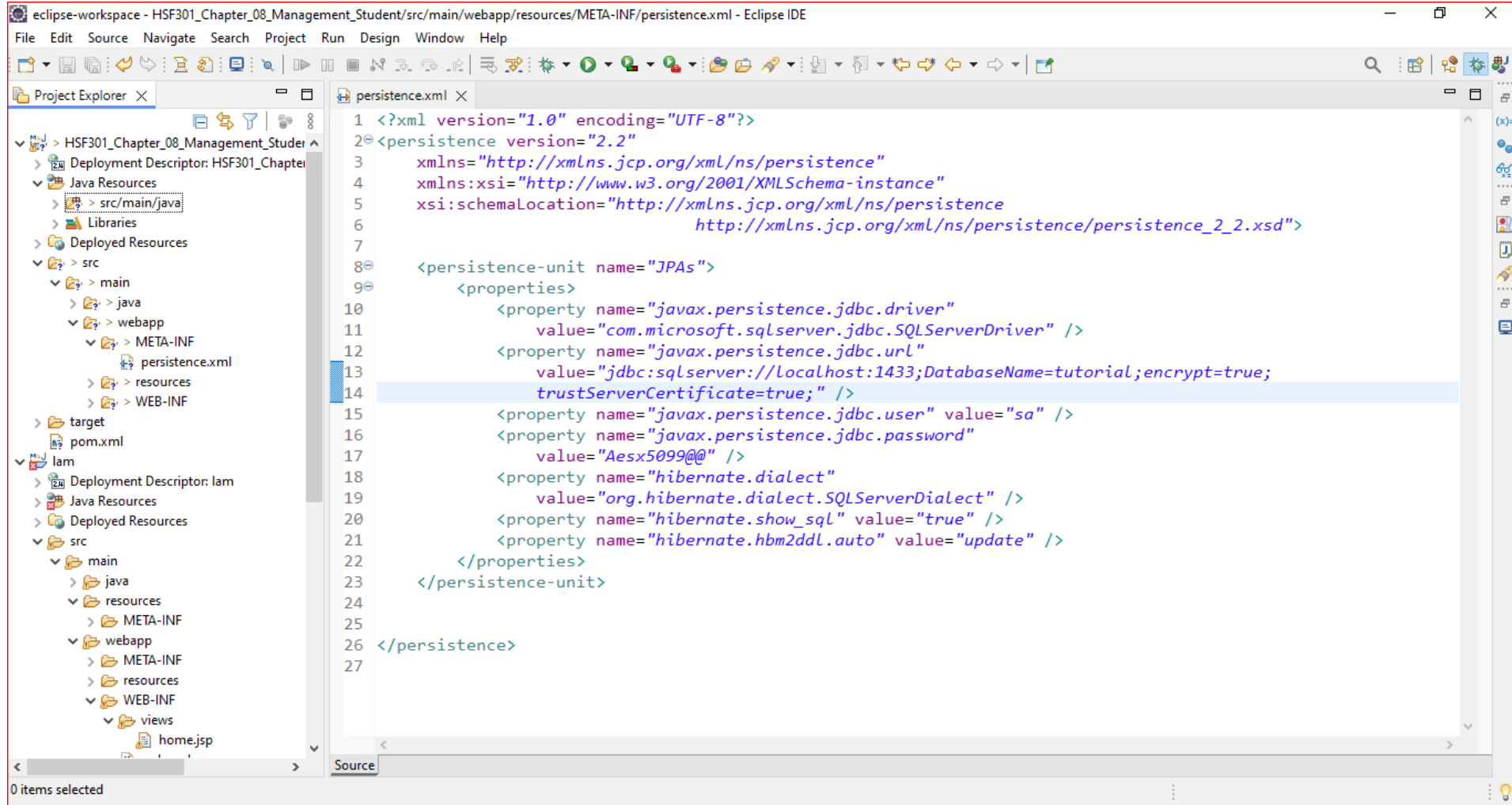


The screenshot shows the Eclipse IDE interface with the 'pom.xml' file open in the editor. The Project Explorer on the left shows the project structure for 'HSF301_Chapter_08_Management_Student'. The editor displays the following XML content:

```
68         <scope>provided</scope>
69     </dependency>
70
71     <!--Add Hibernate!-->
72     <dependency>
73         <groupId>org.eclipse.persistence</groupId>
74         <artifactId>javax.persistence</artifactId>
75         <version>2.2.0</version>
76     </dependency>
77     <dependency>
78         <groupId>org.hibernate</groupId>
79         <artifactId>hibernate-entitymanager</artifactId>
80         <version>5.6.9.Final</version>
81     </dependency>
82     <dependency>
83         <groupId>com.microsoft.sqlserver</groupId>
84         <artifactId>mssql-jdbc</artifactId>
85         <version>11.2.3.jre17</version>
86     </dependency>
87
88     <dependency>
89         <groupId>jstl</groupId>
90         <artifactId>jstl</artifactId>
91         <version>1.2</version>
92     </dependency>
93
94     <!--End DAO!-->
95 </dependencies>
96
97 <repositories>
```

The bottom of the IDE shows the 'pom.xml' tab selected in the 'Overview' view.

12. Create META-INF and persistence.xml File

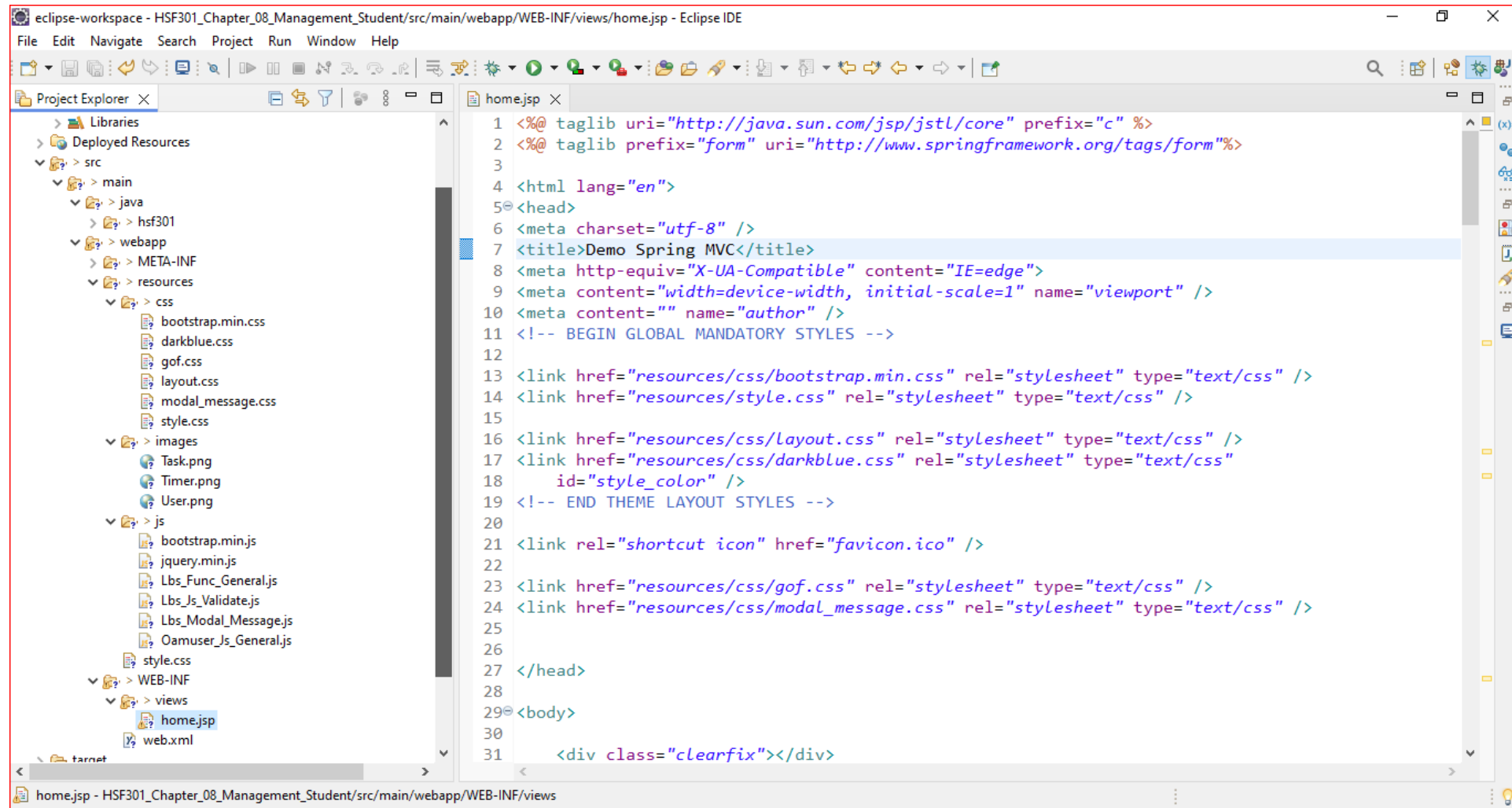


```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/webapp/resources/META-INF/persistence.xml - Eclipse IDE
File Edit Source Navigate Search Project Run Design Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter
    Java Resources
      src/main/java
    Libraries
    Deployed Resources
    src
      main
        java
        webapp
          META-INF
            persistence.xml
          resources
          WEB-INF
    target
    pom.xml
    lam
      Deployment Descriptor: lam
      Java Resources
      Deployed Resources
      src
        main
          java
          resources
          META-INF
          webapp
            META-INF
            resources
            WEB-INF
              views
                home.jsp

persistence.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence version="2.2"
3   xmlns="http://xmlns.jcp.org/xml/ns/persistence"
4   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
6     http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
7
8   <persistence-unit name="JPAs">
9     <properties>
10       <property name="javax.persistence.jdbc.driver"
11         value="com.microsoft.sqlserver.jdbc.SQLServerDriver" />
12       <property name="javax.persistence.jdbc.url"
13         value="jdbc:sqlserver://localhost:1433;DatabaseName=tutorial;encrypt=true;
14         trustServerCertificate=true;" />
15       <property name="javax.persistence.jdbc.user" value="sa" />
16       <property name="javax.persistence.jdbc.password"
17         value="Aesx5099@" />
18       <property name="hibernate.dialect"
19         value="org.hibernate.dialect.SQLServerDialect" />
20       <property name="hibernate.show_sql" value="true" />
21       <property name="hibernate.hbm2ddl.auto" value="update" />
22     </properties>
23   </persistence-unit>
24
25
26 </persistence>
27
```

13. Copy material into project



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure: Libraries, Deployed Resources, src (main, java, hsf301, webapp), META-INF, resources (css, images, js), WEB-INF, views, and web.xml. The main editor window shows the content of home.jsp, which is a JSP file with the following code:

```
1 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2 <%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
3
4 <html lang="en">
5 <head>
6 <meta charset="utf-8" />
7 <title>Demo Spring MVC</title>
8 <meta http-equiv="X-UA-Compatible" content="IE=edge">
9 <meta content="width=device-width, initial-scale=1" name="viewport" />
10 <meta content="" name="author" />
11 <!-- BEGIN GLOBAL MANDATORY STYLES -->
12
13 <link href="resources/css/bootstrap.min.css" rel="stylesheet" type="text/css" />
14 <link href="resources/style.css" rel="stylesheet" type="text/css" />
15
16 <link href="resources/css/layout.css" rel="stylesheet" type="text/css" />
17 <link href="resources/css/darkblue.css" rel="stylesheet" type="text/css"
18     id="style_color" />
19 <!-- END THEME LAYOUT STYLES -->
20
21 <link rel="shortcut icon" href="favicon.ico" />
22
23 <link href="resources/css/gof.css" rel="stylesheet" type="text/css" />
24 <link href="resources/css/modal_message.css" rel="stylesheet" type="text/css" />
25
26 </head>
27
28 <body>
29
30     <div class="clearfix"></div>
```

14. Result

localhost:8080/HSF301_Chapter_08_Management_Student/

FPT DesignPattern Microbit E-Learning & Educa... Maqueen Robo t-... Perplexity Consensus: AI Searc... Robot hormiga - Yo... Động cơ DC giảm t... All Bookmarks

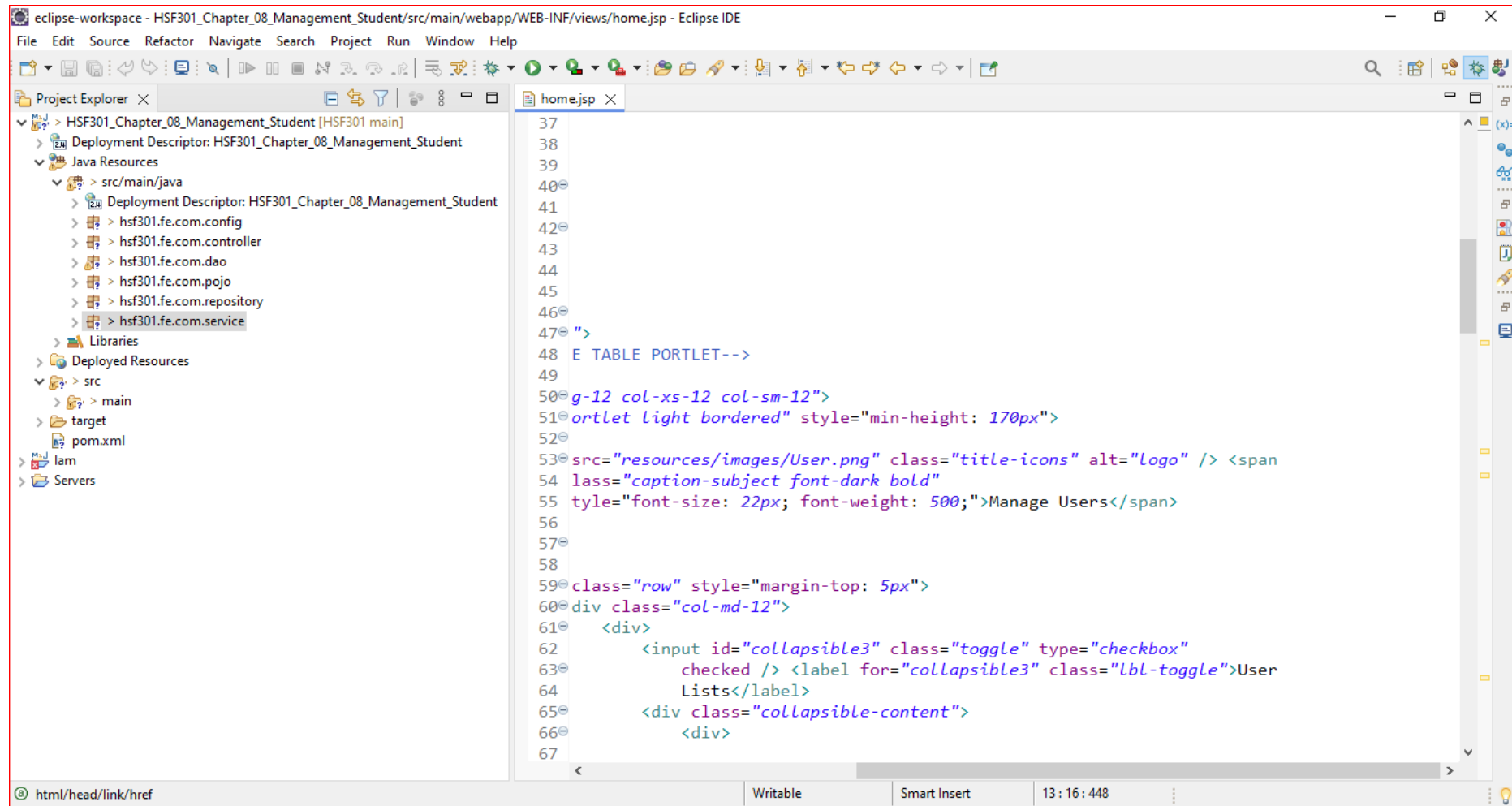
Manage Users

▼ User Lists

Choose	ID	First Name	Last Name	Mark
--------	----	------------	-----------	------

Key Name	Key Value
ID	
First Name	
Last Name	
Mark	

15. Build the Architecture

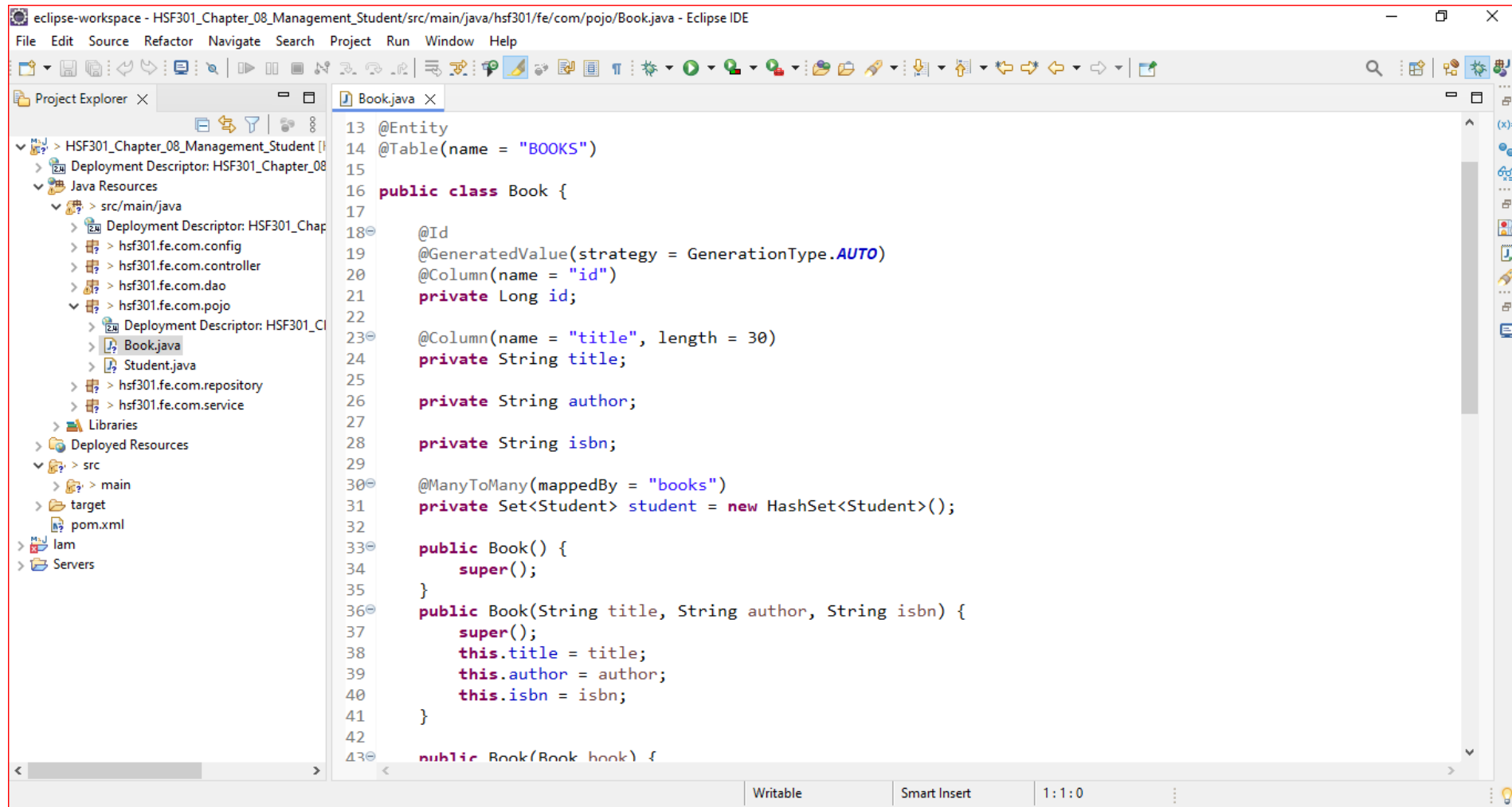


The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the 'home.jsp' file with the following code:

```
37
38
39
40
41
42
43
44
45
46
47 ">
48 E TABLE PORTLET-->
49
50 g-12 col-xs-12 col-sm-12">
51 ortlet light bordered" style="min-height: 170px">
52
53 src="resources/images/User.png" class="title-icons" alt="Logo" /> <span
54 lass="caption-subject font-dark bold"
55 tyle="font-size: 22px; font-weight: 500;">Manage Users</span>
56
57
58
59 class="row" style="margin-top: 5px">
60 div class="col-md-12">
61 <div>
62 <input id="collapsible3" class="toggle" type="checkbox"
63 checked /> <label for="collapsible3" class="lbl-toggle">User
64 Lists</label>
65 <div class="collapsible-content">
66 <div>
67
```

The status bar at the bottom indicates the current file is 'html/head/link/href', it is 'Writable', 'Smart Insert' is enabled, and the cursor is at line 13, column 16, character 448.

16. Create Books in Pojo

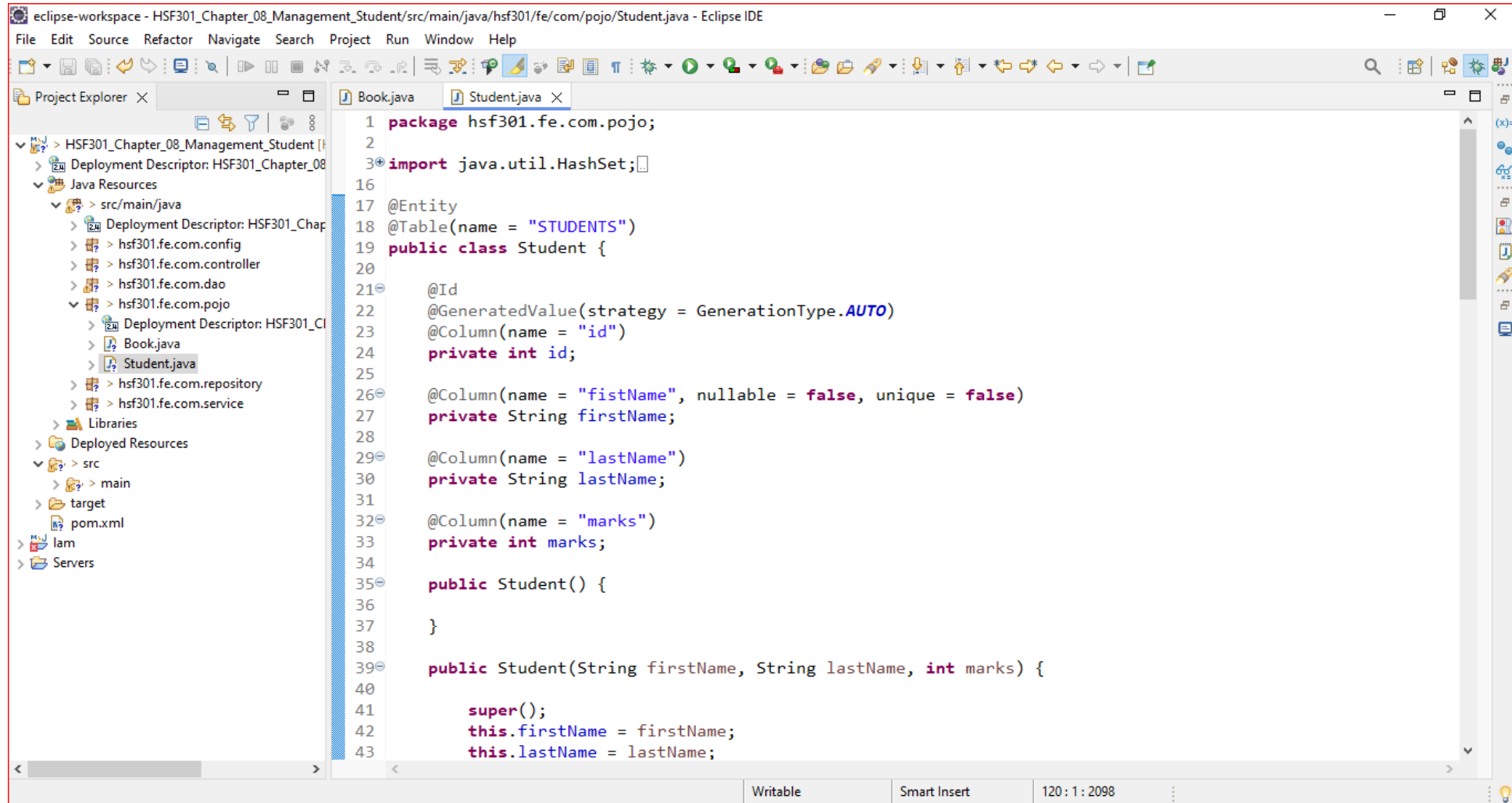


```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/pojo/Book.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter_08
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_Chapter_08
        hsf301.fe.com.config
        hsf301.fe.com.controller
        hsf301.fe.com.dao
        hsf301.fe.com.pojo
          Deployment Descriptor: HSF301_Chapter_08
          Book.java
          Student.java
        hsf301.fe.com.repository
        hsf301.fe.com.service
      Libraries
    Deployed Resources
    src
      main
      target
    pom.xml
    lam
    Servers

Book.java
13 @Entity
14 @Table(name = "BOOKS")
15
16 public class Book {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.AUTO)
20     @Column(name = "id")
21     private Long id;
22
23     @Column(name = "title", length = 30)
24     private String title;
25
26     private String author;
27
28     private String isbn;
29
30     @ManyToMany(mappedBy = "books")
31     private Set<Student> student = new HashSet<Student>();
32
33     public Book() {
34         super();
35     }
36
37     public Book(String title, String author, String isbn) {
38         super();
39         this.title = title;
40         this.author = author;
41         this.isbn = isbn;
42     }
43
44     public Book(Book book) {
```

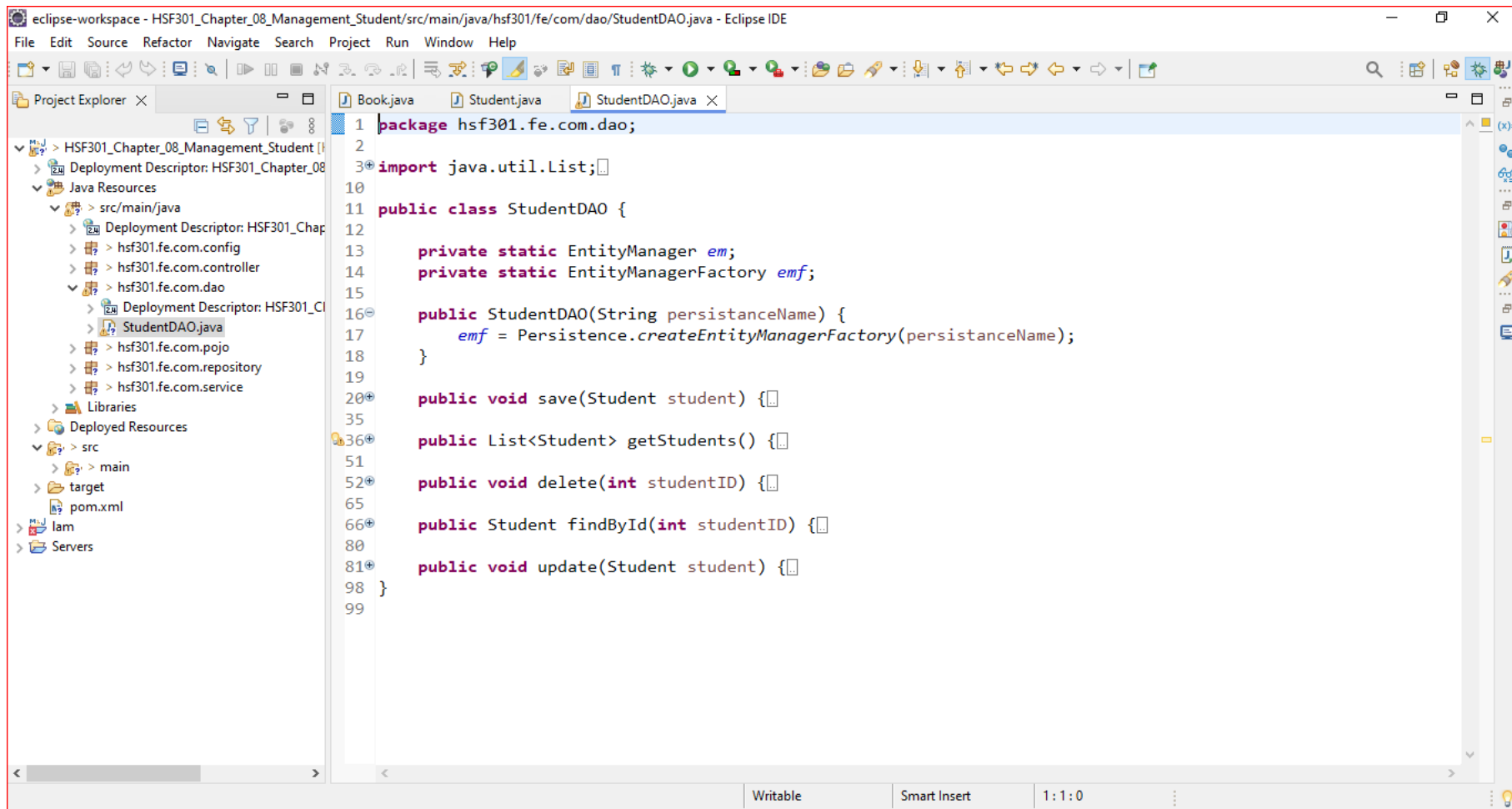
17. Create Students in Pojo



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the code for 'Student.java'.

```
1 package hsf301.fe.com.pojo;
2
3 import java.util.HashSet;
4
5
6
7
8
9
10
11
12
13
14
15
16
17 @Entity
18 @Table(name = "STUDENTS")
19 public class Student {
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.AUTO)
23     @Column(name = "id")
24     private int id;
25
26     @Column(name = "firstName", nullable = false, unique = false)
27     private String firstName;
28
29     @Column(name = "lastName")
30     private String lastName;
31
32     @Column(name = "marks")
33     private int marks;
34
35     public Student() {
36
37     }
38
39     public Student(String firstName, String lastName, int marks) {
40
41         super();
42         this.firstName = firstName;
43         this.lastName = lastName;
```

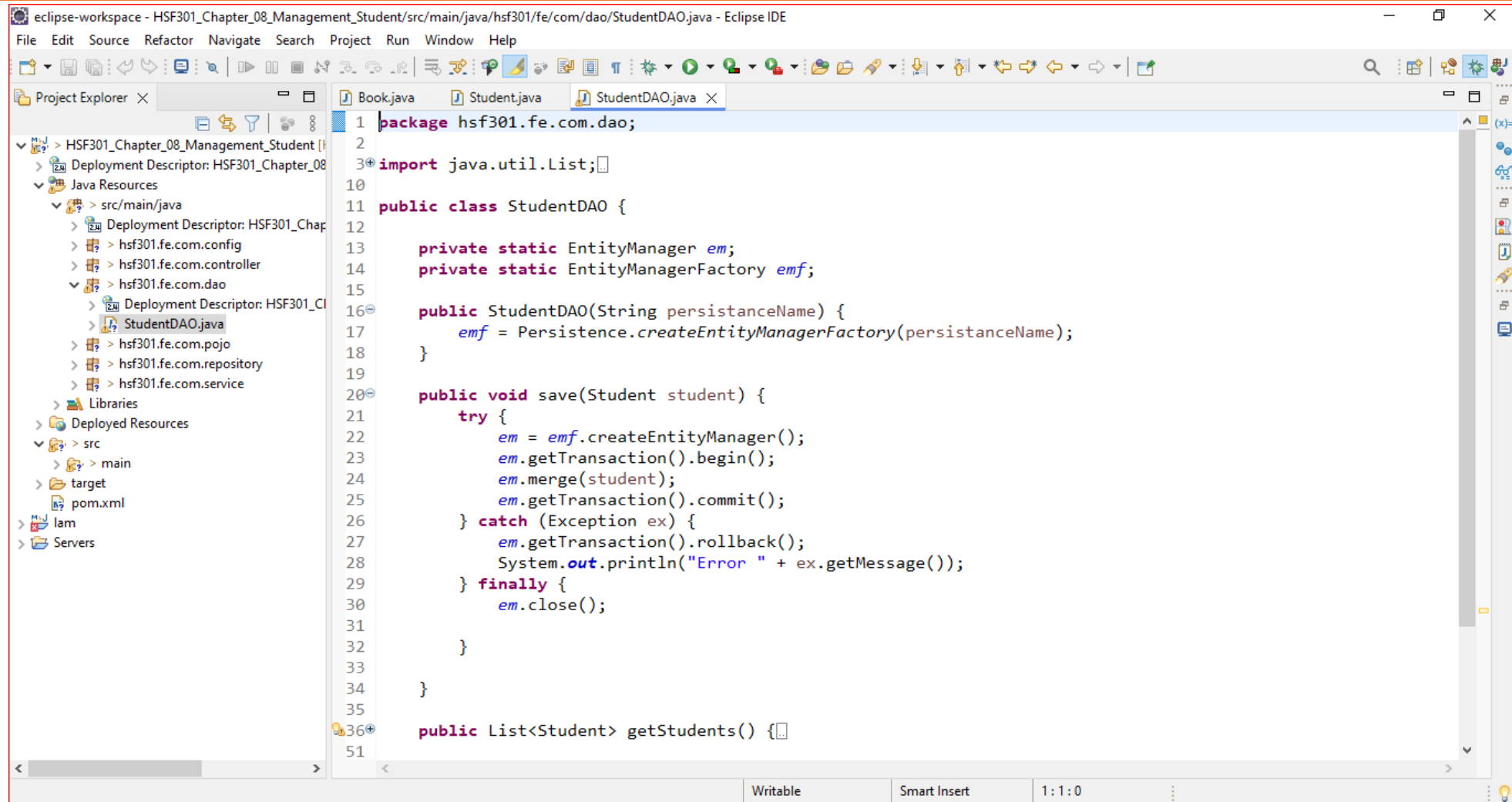
18. Create StudentDAO



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the code for 'StudentDAO.java'. The code defines a package, imports List, and implements the StudentDAO interface with methods for save, getStudents, delete, findById, and update.

```
1 package hsf301.fe.com.dao;
2
3 import java.util.List;
4
5
6
7
8
9
10
11 public class StudentDAO {
12
13     private static EntityManager em;
14     private static EntityManagerFactory emf;
15
16     public StudentDAO(String persistenceName) {
17         emf = Persistence.createEntityManagerFactory(persistenceName);
18     }
19
20     public void save(Student student) {}
21
22     public List<Student> getStudents() {}
23
24     public void delete(int studentID) {}
25
26     public Student findById(int studentID) {}
27
28     public void update(Student student) {}
29 }
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

19. Save Student in StudentDAO

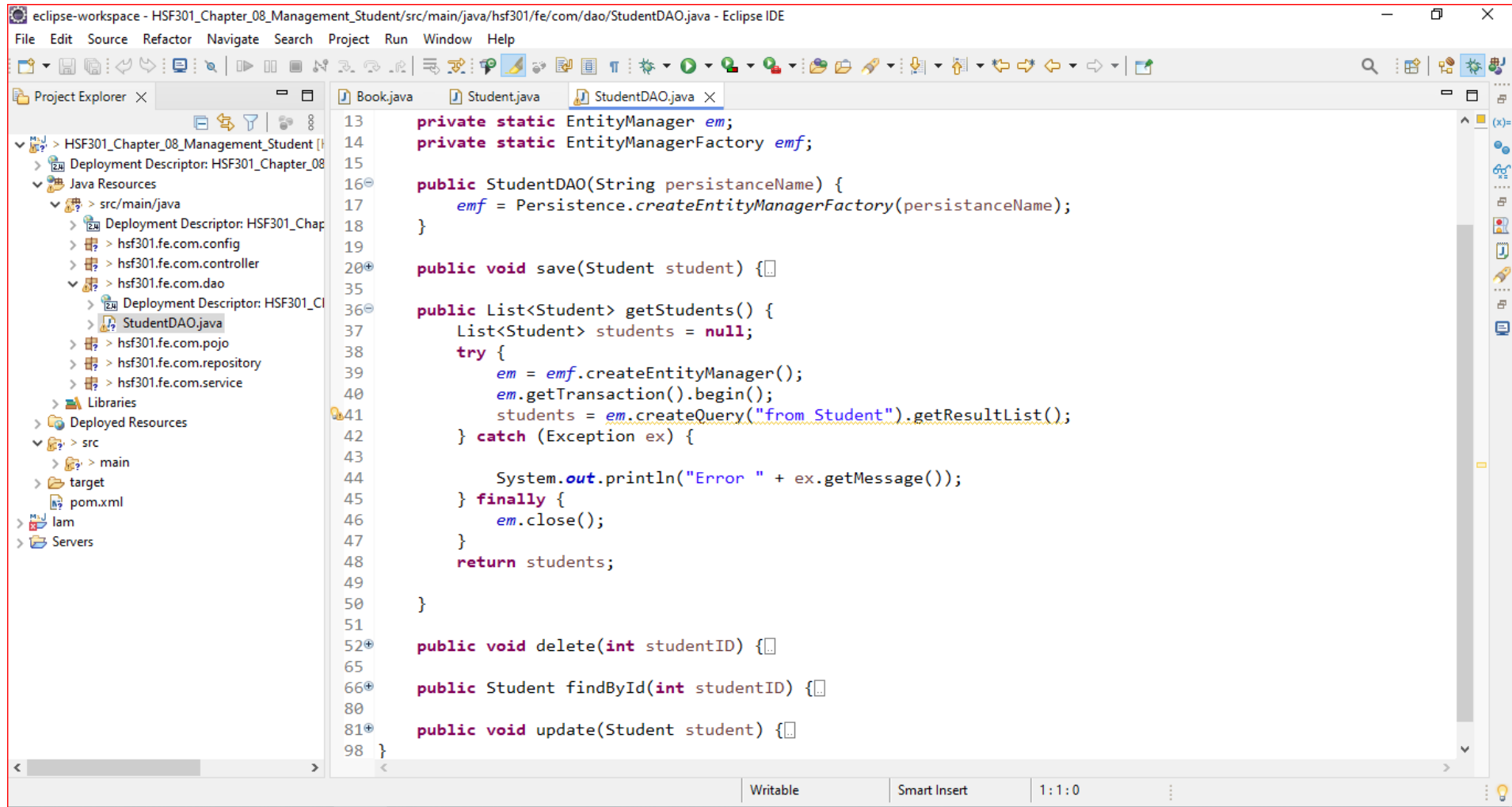


```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/dao/StudentDAO.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter_08
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_Chapter_08
        hsf301.fe.com.config
        hsf301.fe.com.controller
        hsf301.fe.com.dao
          StudentDAO.java
        hsf301.fe.com.pojo
        hsf301.fe.com.repository
        hsf301.fe.com.service
      Libraries
      Deployed Resources
    src
      main
      target
    pom.xml
    lam
    Servers

1 package hsf301.fe.com.dao;
2
3 import java.util.List;
4
5
6
7
8
9
10
11 public class StudentDAO {
12
13     private static EntityManager em;
14     private static EntityManagerFactory emf;
15
16     public StudentDAO(String persistenceName) {
17         emf = Persistence.createEntityManagerFactory(persistenceName);
18     }
19
20     public void save(Student student) {
21         try {
22             em = emf.createEntityManager();
23             em.getTransaction().begin();
24             em.merge(student);
25             em.getTransaction().commit();
26         } catch (Exception ex) {
27             em.getTransaction().rollback();
28             System.out.println("Error " + ex.getMessage());
29         } finally {
30             em.close();
31         }
32     }
33
34 }
35
36 public List<Student> getStudents() {
37
38 }
39
40
41
42
43
44
45
46
47
48
49
50
51
```

20. Get All Student in StudentDAO

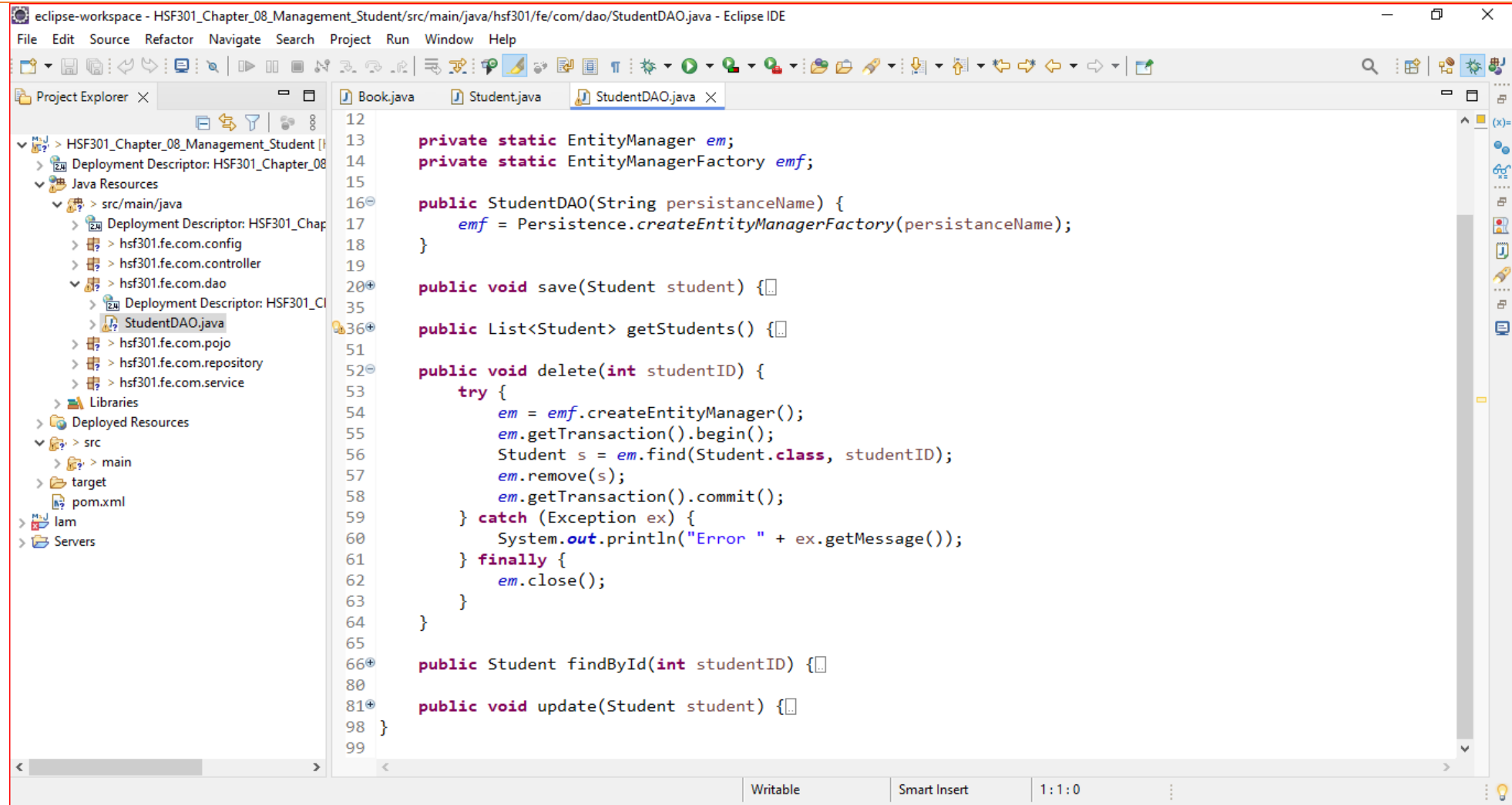


```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/dao/StudentDAO.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter_08
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_Chapter_08
        hsf301.fe.com.config
        hsf301.fe.com.controller
        hsf301.fe.com.dao
          StudentDAO.java
        hsf301.fe.com.pojo
        hsf301.fe.com.repository
        hsf301.fe.com.service
      Libraries
    Deployed Resources
      src
        main
      target
    pom.xml
    lam
    Servers

13 private static EntityManager em;
14 private static EntityManagerFactory emf;
15
16 public StudentDAO(String persistenceName) {
17     emf = Persistence.createEntityManagerFactory(persistenceName);
18 }
19
20 public void save(Student student) {}
21
22 public List<Student> getStudents() {
23     List<Student> students = null;
24     try {
25         em = emf.createEntityManager();
26         em.getTransaction().begin();
27         students = em.createQuery("from Student").getResultList();
28     } catch (Exception ex) {
29         System.out.println("Error " + ex.getMessage());
30     } finally {
31         em.close();
32     }
33     return students;
34 }
35
36 public void delete(int studentID) {}
37
38 public Student findById(int studentID) {}
39
40 public void update(Student student) {}
41 }
```

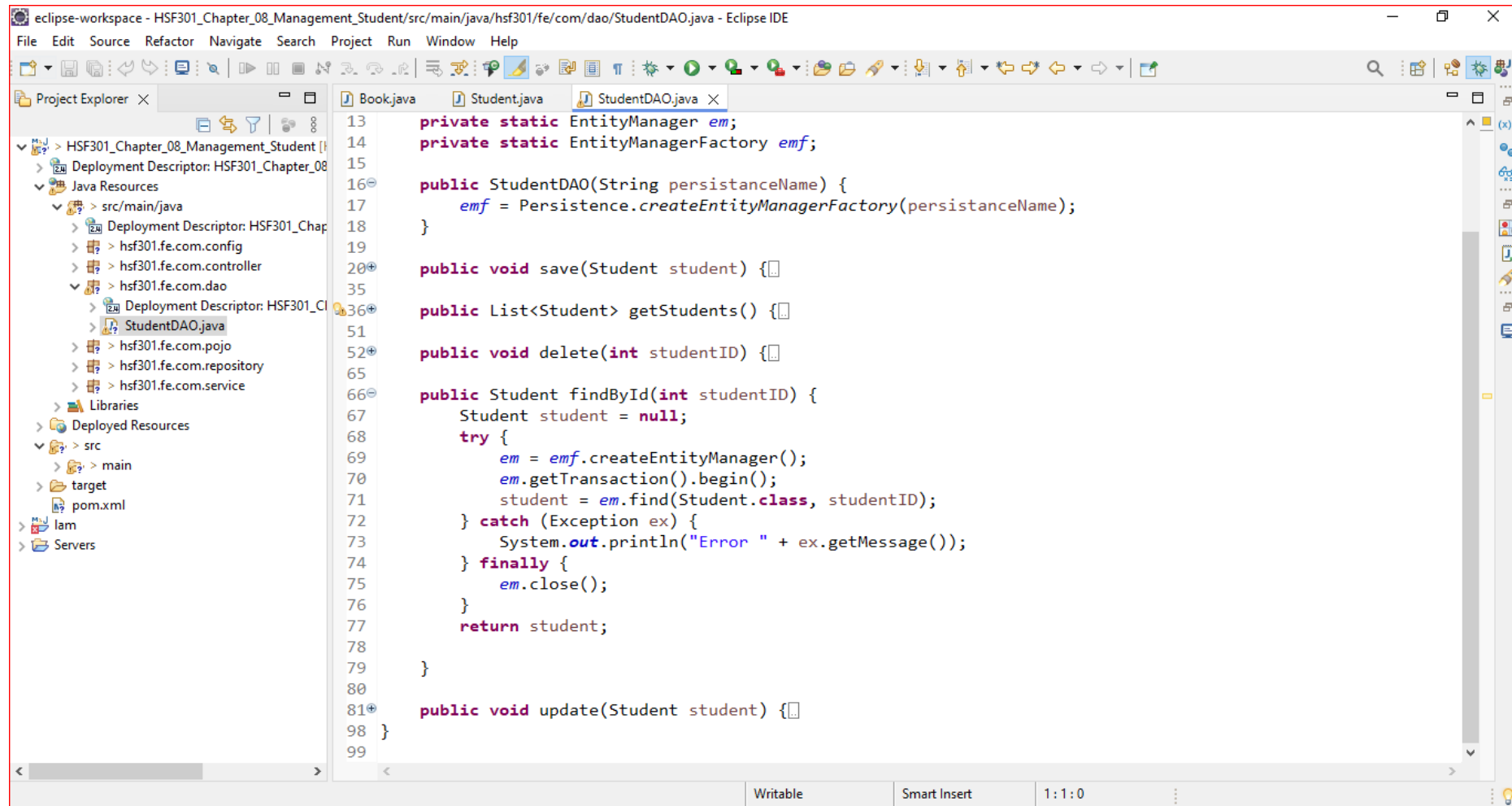
21. Delete Student in StudentDAO



The screenshot shows the Eclipse IDE with the file `StudentDAO.java` open. The code implements the `delete` method, which uses JPA's `EntityManager` to remove a student from the database. The `delete` method is part of the `StudentDAO` class, which also includes methods for saving, finding, and updating students. The `delete` method uses a `try-catch-finally` block to ensure the `EntityManager` is properly closed after the operation.

```
12
13 private static EntityManager em;
14 private static EntityManagerFactory emf;
15
16 public StudentDAO(String persistenceName) {
17     emf = Persistence.createEntityManagerFactory(persistenceName);
18 }
19
20 public void save(Student student) {}
21
22 public List<Student> getStudents() {}
23
24 public void delete(int studentID) {
25     try {
26         em = emf.createEntityManager();
27         em.getTransaction().begin();
28         Student s = em.find(Student.class, studentID);
29         em.remove(s);
30         em.getTransaction().commit();
31     } catch (Exception ex) {
32         System.out.println("Error " + ex.getMessage());
33     } finally {
34         em.close();
35     }
36 }
37
38 public Student findById(int studentID) {}
39
40 public void update(Student student) {}
41 }
```

22. Find A Student in StudentDAO



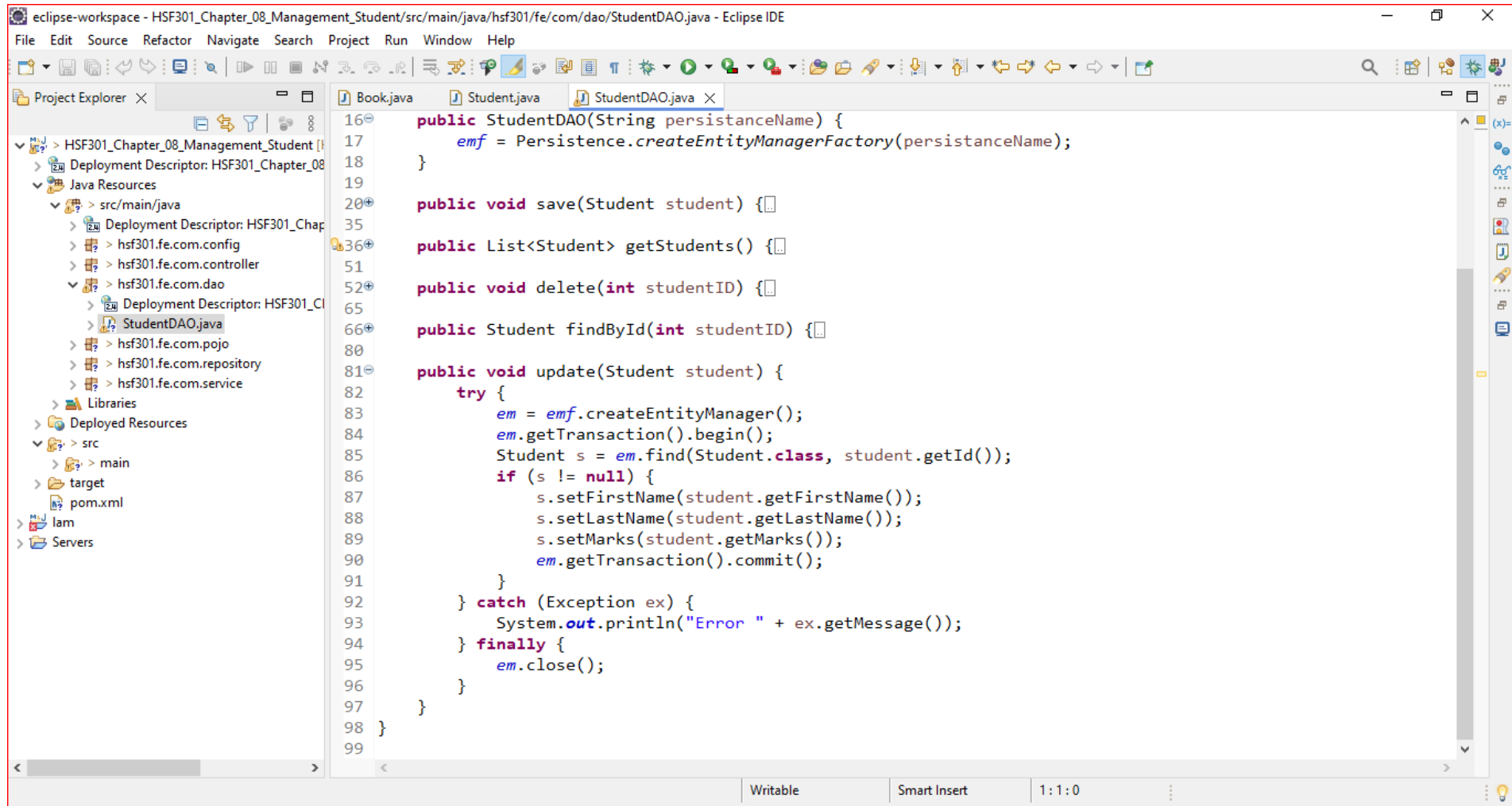
```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/dao/StudentDAO.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter_08
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_Chapter_08
        hsf301.fe.com.config
        hsf301.fe.com.controller
        hsf301.fe.com.dao
          StudentDAO.java
        hsf301.fe.com.pojo
        hsf301.fe.com.repository
        hsf301.fe.com.service
      Libraries
      Deployed Resources
      src
        main
      target
      pom.xml
    lam
    Servers

13 private static EntityManager em;
14 private static EntityManagerFactory emf;
15
16 public StudentDAO(String persistenceName) {
17     emf = Persistence.createEntityManagerFactory(persistenceName);
18 }
19
20 public void save(Student student) {}
35
36 public List<Student> getStudents() {}
51
52 public void delete(int studentID) {}
65
66 public Student findById(int studentID) {
67     Student student = null;
68     try {
69         em = emf.createEntityManager();
70         em.getTransaction().begin();
71         student = em.find(Student.class, studentID);
72     } catch (Exception ex) {
73         System.out.println("Error " + ex.getMessage());
74     } finally {
75         em.close();
76     }
77     return student;
78 }
79
80
81 public void update(Student student) {}
98 }
99

Writable Smart Insert 1:1:0
```

23. Update a Student in StudentDAO

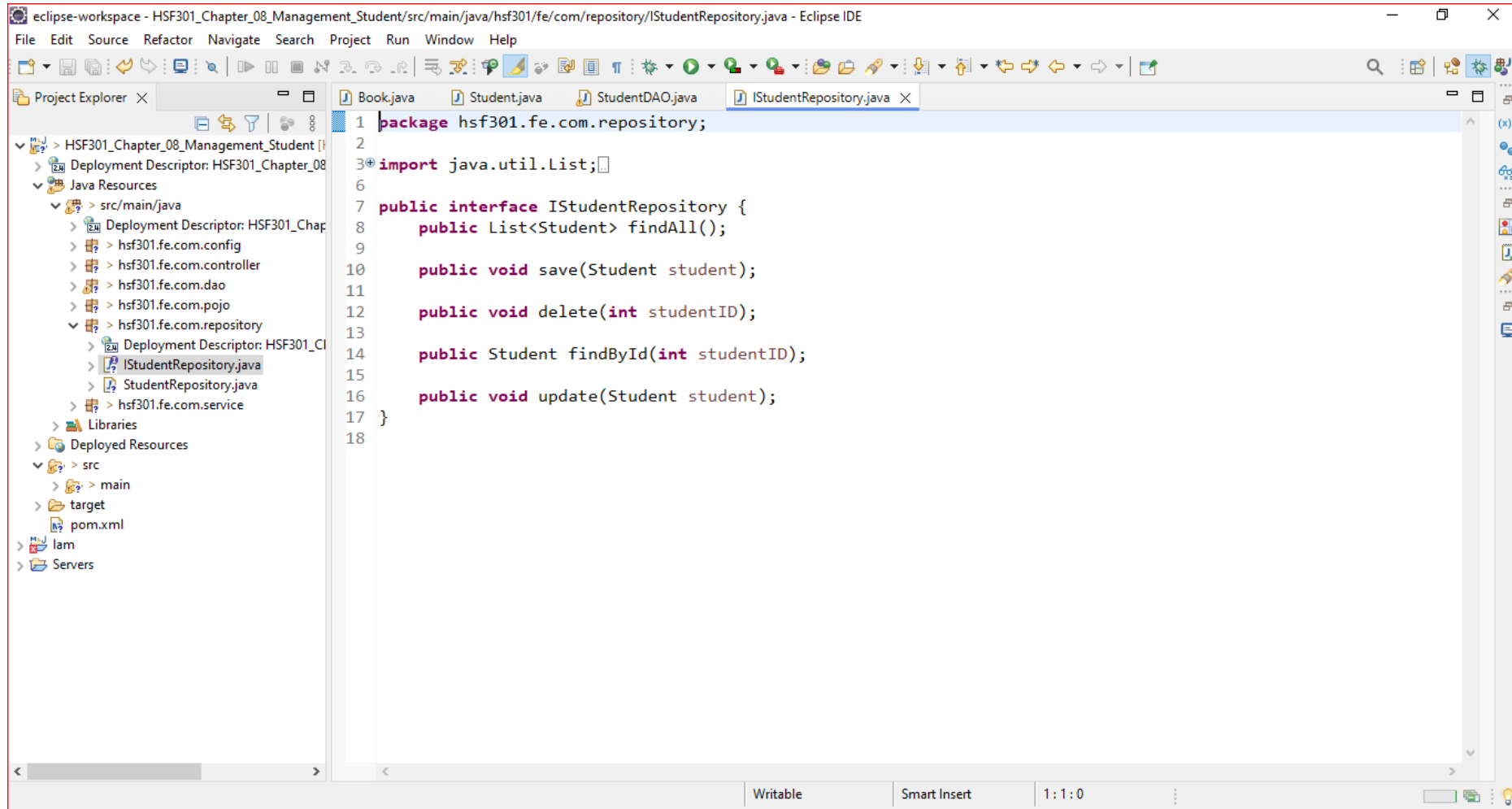


The screenshot shows the Eclipse IDE with the following components:

- Project Explorer:** Displays the project structure for 'HSF301_Chapter_08_Management_Student'. The 'src/main/java' directory is expanded, showing the package 'hsf301.fe.com.dao' containing 'StudentDAO.java'.
- Editor:** Shows the code for 'StudentDAO.java'. The 'update' method is implemented as follows:

```
16 public StudentDAO(String persistenceName) {  
17     emf = Persistence.createEntityManagerFactory(persistenceName);  
18 }  
19  
20 public void save(Student student) {}  
21  
36 public List<Student> getStudents() {}  
37  
51 public void delete(int studentID) {}  
52  
65 public Student findById(int studentID) {}  
66  
80 public void update(Student student) {  
81     try {  
82         em = emf.createEntityManager();  
83         em.getTransaction().begin();  
84         Student s = em.find(Student.class, student.getId());  
85         if (s != null) {  
86             s.setFirstName(student.getFirstName());  
87             s.setLastName(student.getLastName());  
88             s.setMarks(student.getMarks());  
89             em.getTransaction().commit();  
90         }  
91     } catch (Exception ex) {  
92         System.out.println("Error " + ex.getMessage());  
93     } finally {  
94         em.close();  
95     }  
96 }  
97 }  
98 }  
99 }
```
- Bottom Bar:** Shows 'Writable', 'Smart Insert', and '1:1:0'.

24. Create IStudentRepository

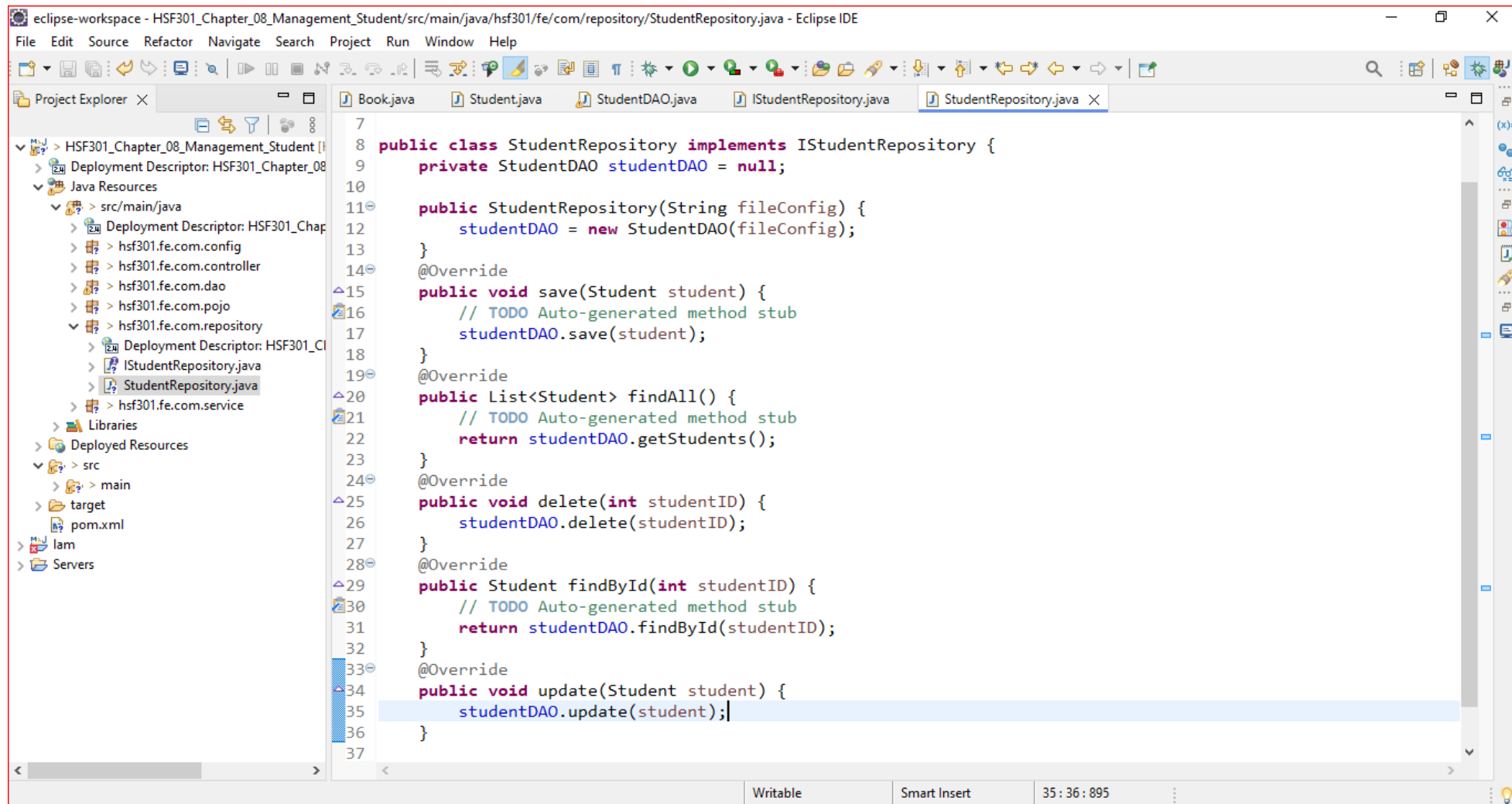


The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The 'src/main/java' directory is expanded, showing the 'hsf301.fe.com.repository' package. The 'IStudentRepository.java' file is selected. The main editor window shows the following code:

```
1 package hsf301.fe.com.repository;
2
3 import java.util.List;
4
5
6
7 public interface IStudentRepository {
8     public List<Student> findAll();
9
10    public void save(Student student);
11
12    public void delete(int studentID);
13
14    public Student findById(int studentID);
15
16    public void update(Student student);
17 }
18
```

The status bar at the bottom indicates 'Writable', 'Smart Insert', and '1:1:0'.

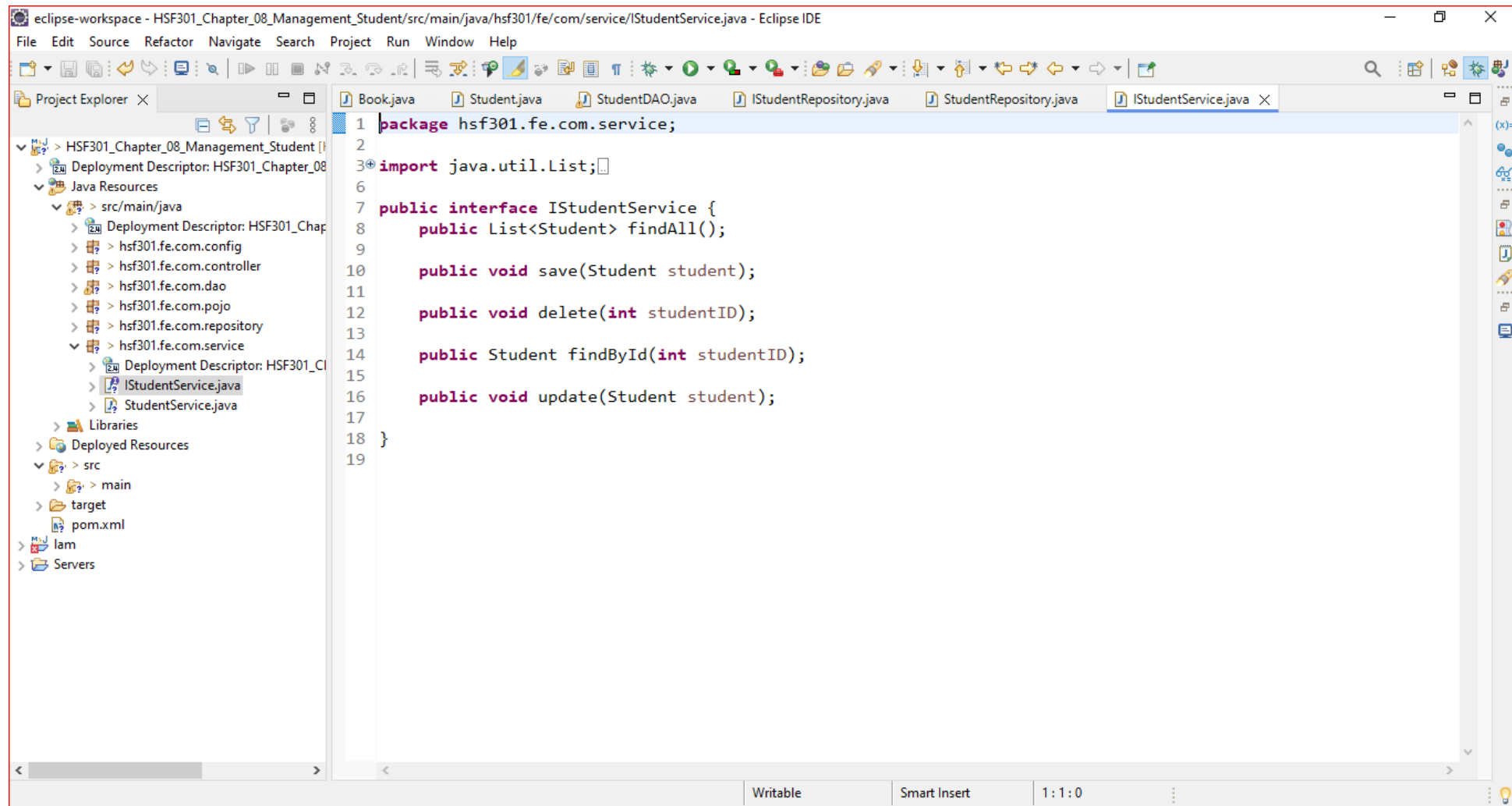
25. Create StudentRepository



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the code for 'StudentRepository.java', which implements the 'IStudentRepository' interface. The code includes a private field 'studentDAO' and several methods: 'StudentRepository(String fileConfig)', 'save(Student student)', 'findAll()', 'delete(int studentID)', 'findById(int studentID)', and 'update(Student student)'. The 'update' method is currently selected and highlighted in blue.

```
7  
8 public class StudentRepository implements IStudentRepository {  
9     private StudentDAO studentDAO = null;  
10  
11     public StudentRepository(String fileConfig) {  
12         studentDAO = new StudentDAO(fileConfig);  
13     }  
14     @Override  
15     public void save(Student student) {  
16         // TODO Auto-generated method stub  
17         studentDAO.save(student);  
18     }  
19     @Override  
20     public List<Student> findAll() {  
21         // TODO Auto-generated method stub  
22         return studentDAO.getStudents();  
23     }  
24     @Override  
25     public void delete(int studentID) {  
26         studentDAO.delete(studentID);  
27     }  
28     @Override  
29     public Student findById(int studentID) {  
30         // TODO Auto-generated method stub  
31         return studentDAO.findById(studentID);  
32     }  
33     @Override  
34     public void update(Student student) {  
35         studentDAO.update(student);  
36     }  
37 }
```

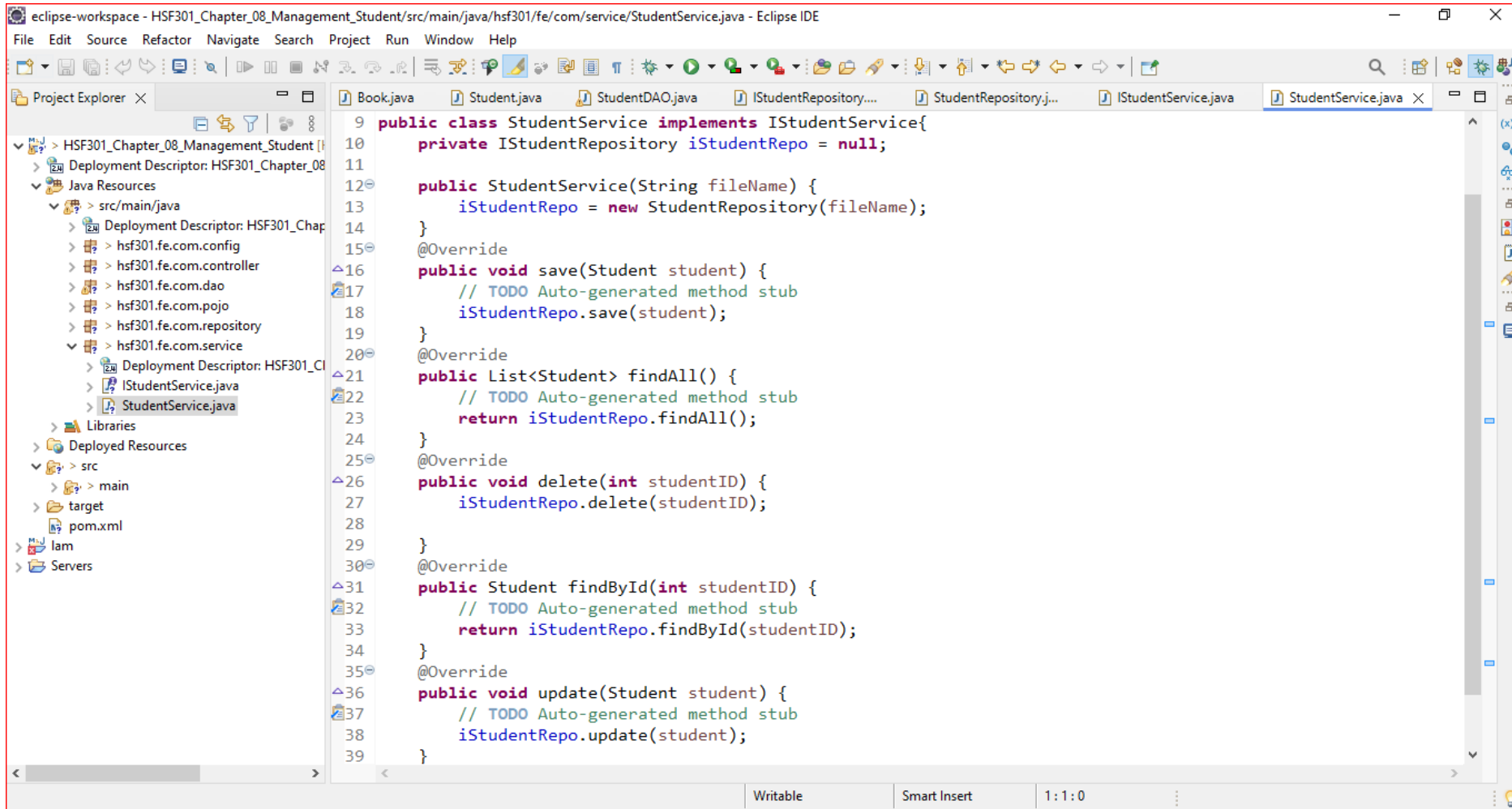
26. Create IStudentService



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the code for 'IStudentService.java'. The code defines a package, imports a utility class, and declares an interface with several methods.

```
1 package hsf301.fe.com.service;
2
3 import java.util.List;
4
5
6
7 public interface IStudentService {
8     public List<Student> findAll();
9
10    public void save(Student student);
11
12    public void delete(int studentID);
13
14    public Student findById(int studentID);
15
16    public void update(Student student);
17
18 }
19
```

27. Create StudentService



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The 'src/main/java' directory is expanded, showing the package 'hsf301.fe.com.service'. The 'StudentService.java' file is selected. The main editor window shows the code for 'StudentService' which implements 'IStudentService'. The code includes a private field 'iStudentRepo' and several methods: 'StudentService(String fileName)', 'save(Student student)', 'findAll()', 'delete(int studentID)', 'findById(int studentID)', and 'update(Student student)'. Each method contains a TODO comment indicating it is an auto-generated stub.

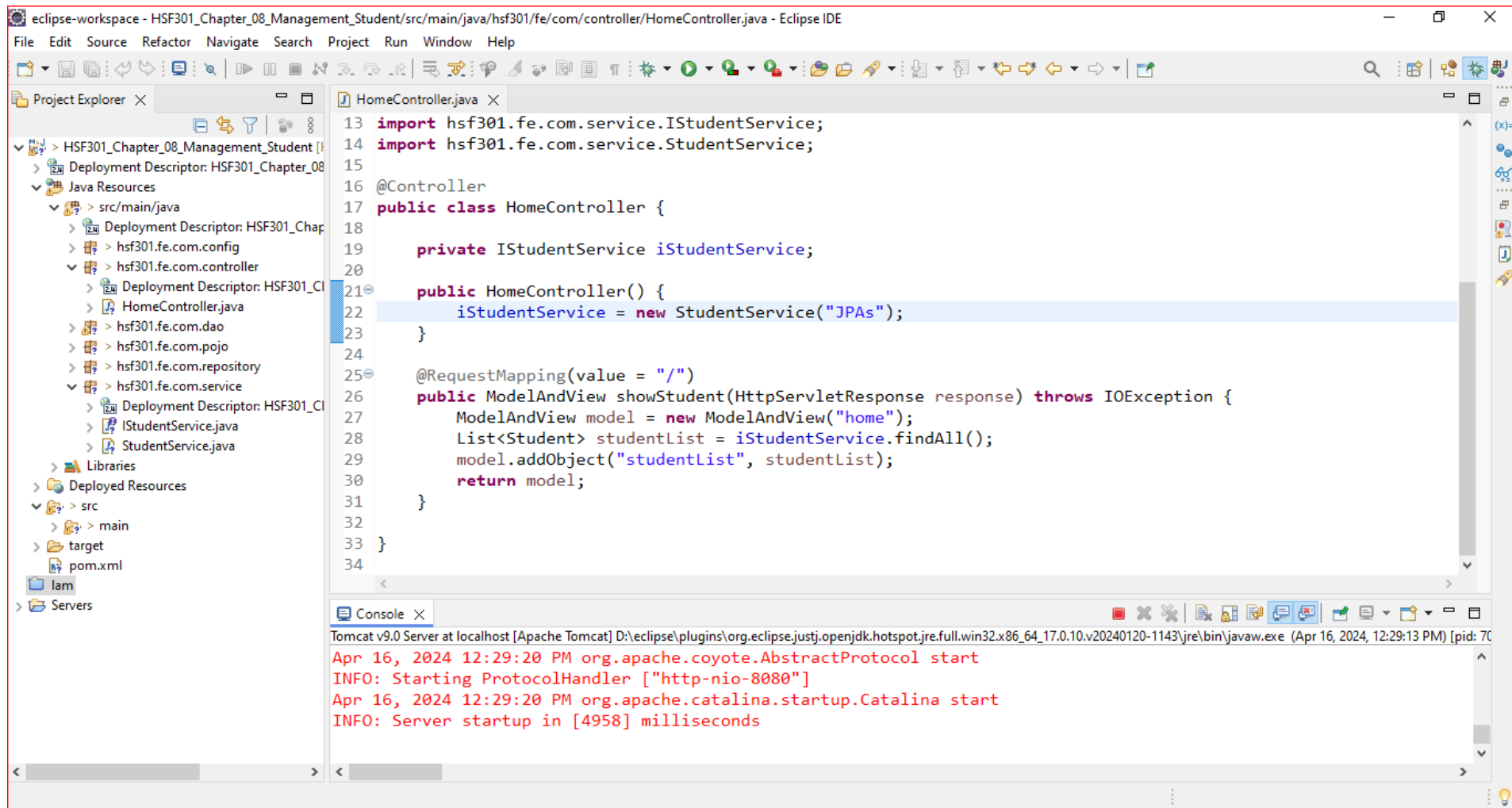
```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/service/StudentService.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter_08
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_Chapter_08
        hsf301.fe.com.config
        hsf301.fe.com.controller
        hsf301.fe.com.dao
        hsf301.fe.com.pojo
        hsf301.fe.com.repository
        hsf301.fe.com.service
          Deployment Descriptor: HSF301_Chapter_08
          IStudentService.java
          StudentService.java
        Libraries
      Deployed Resources
    src
      main
      target
      pom.xml
    lam
    Servers

Book.java Student.java StudentDAO.java IStudentRepository.java StudentRepository.java IStudentService.java StudentService.java
9 public class StudentService implements IStudentService{
10     private IStudentRepository iStudentRepo = null;
11
12     public StudentService(String fileName) {
13         iStudentRepo = new StudentRepository(fileName);
14     }
15     @Override
16     public void save(Student student) {
17         // TODO Auto-generated method stub
18         iStudentRepo.save(student);
19     }
20     @Override
21     public List<Student> findAll() {
22         // TODO Auto-generated method stub
23         return iStudentRepo.findAll();
24     }
25     @Override
26     public void delete(int studentID) {
27         iStudentRepo.delete(studentID);
28     }
29
30     @Override
31     public Student findById(int studentID) {
32         // TODO Auto-generated method stub
33         return iStudentRepo.findById(studentID);
34     }
35     @Override
36     public void update(Student student) {
37         // TODO Auto-generated method stub
38         iStudentRepo.update(student);
39 }
```

Writable Smart Insert 1:1:0

28. Edit HomeController.java



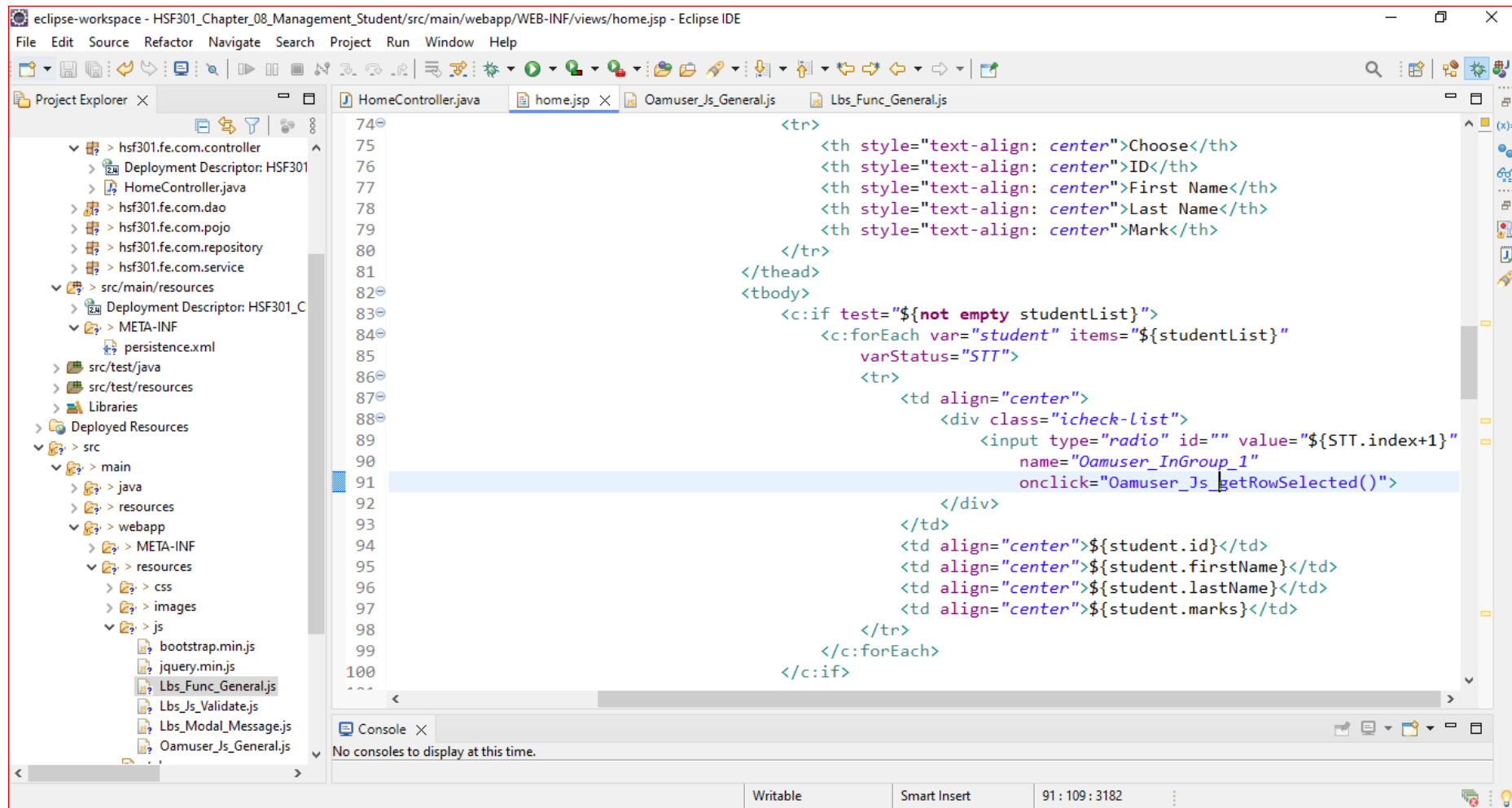
The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'HSF301_Chapter_08_Management_Student'. The main editor window shows the 'HomeController.java' file with the following code:

```
13 import hsf301.fe.com.service.IStudentService;
14 import hsf301.fe.com.service.StudentService;
15
16 @Controller
17 public class HomeController {
18
19     private IStudentService iStudentService;
20
21     public HomeController() {
22         iStudentService = new StudentService("JPAs");
23     }
24
25     @RequestMapping(value = "/")
26     public ModelAndView showStudent(HttpServletResponse response) throws IOException {
27         ModelAndView model = new ModelAndView("home");
28         List<Student> studentList = iStudentService.findAll();
29         model.addObject("studentList", studentList);
30         return model;
31     }
32 }
33
34
```

The Console window at the bottom shows the following output:

```
Tomcat v9.0 Server at localhost [Apache Tomcat] D:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.10.v20240120-1143\jre\bin\javaw.exe (Apr 16, 2024, 12:29:13 PM) [pid: 70]
Apr 16, 2024 12:29:20 PM org.apache.coyote.AbstractProtocol start
INFO: Starting ProtocolHandler ["http-nio-8080"]
Apr 16, 2024 12:29:20 PM org.apache.catalina.startup.Catalina start
INFO: Server startup in [4958] milliseconds
```

28. Edit Home.jsp



```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/webapp/WEB-INF/views/home.jsp - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  hsf301.fe.com.controller
  Deployment Descriptor: HSF301
  HomeController.java
  hsf301.fe.com.dao
  hsf301.fe.com.pojo
  hsf301.fe.com.repository
  hsf301.fe.com.service
  src/main/resources
  Deployment Descriptor: HSF301_C
  META-INF
  persistence.xml
  src/test/java
  src/test/resources
  Libraries
  Deployed Resources
  src
  main
  java
  resources
  webapp
  META-INF
  resources
  css
  images
  js
  bootstrap.min.js
  jquery.min.js
  Lbs_Func_General.js
  Lbs_Validate.js
  Lbs_Modal_Message.js
  Oamuser_Json_General.js

HomeController.java
home.jsp
Oamuser_Json_General.js
Lbs_Func_General.js

74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

<tr>
  <th style="text-align: center">Choose</th>
  <th style="text-align: center">ID</th>
  <th style="text-align: center">First Name</th>
  <th style="text-align: center">Last Name</th>
  <th style="text-align: center">Mark</th>
</tr>
</thead>
<tbody>
  <c:if test="${not empty studentList}">
    <c:forEach var="student" items="${studentList}">
      varStatus="STT">
        <tr>
          <td align="center">
            <div class="icheck-list">
              <input type="radio" id="" value="${STT.index+1}"
                name="Oamuser_InGroup_1"
                onclick="Oamuser_Json_GetRowSelected()">
            </div>
          </td>
          <td align="center">${student.id}</td>
          <td align="center">${student.firstName}</td>
          <td align="center">${student.lastName}</td>
          <td align="center">${student.marks}</td>
        </tr>
      </c:forEach>
    </c:if>
  </tbody>
</table>
```

Console

No consoles to display at this time.

Writable Smart Insert 91 : 109 : 3182

19. Result

localhost:8080/HSF301_Chapter_08_Management_Student/

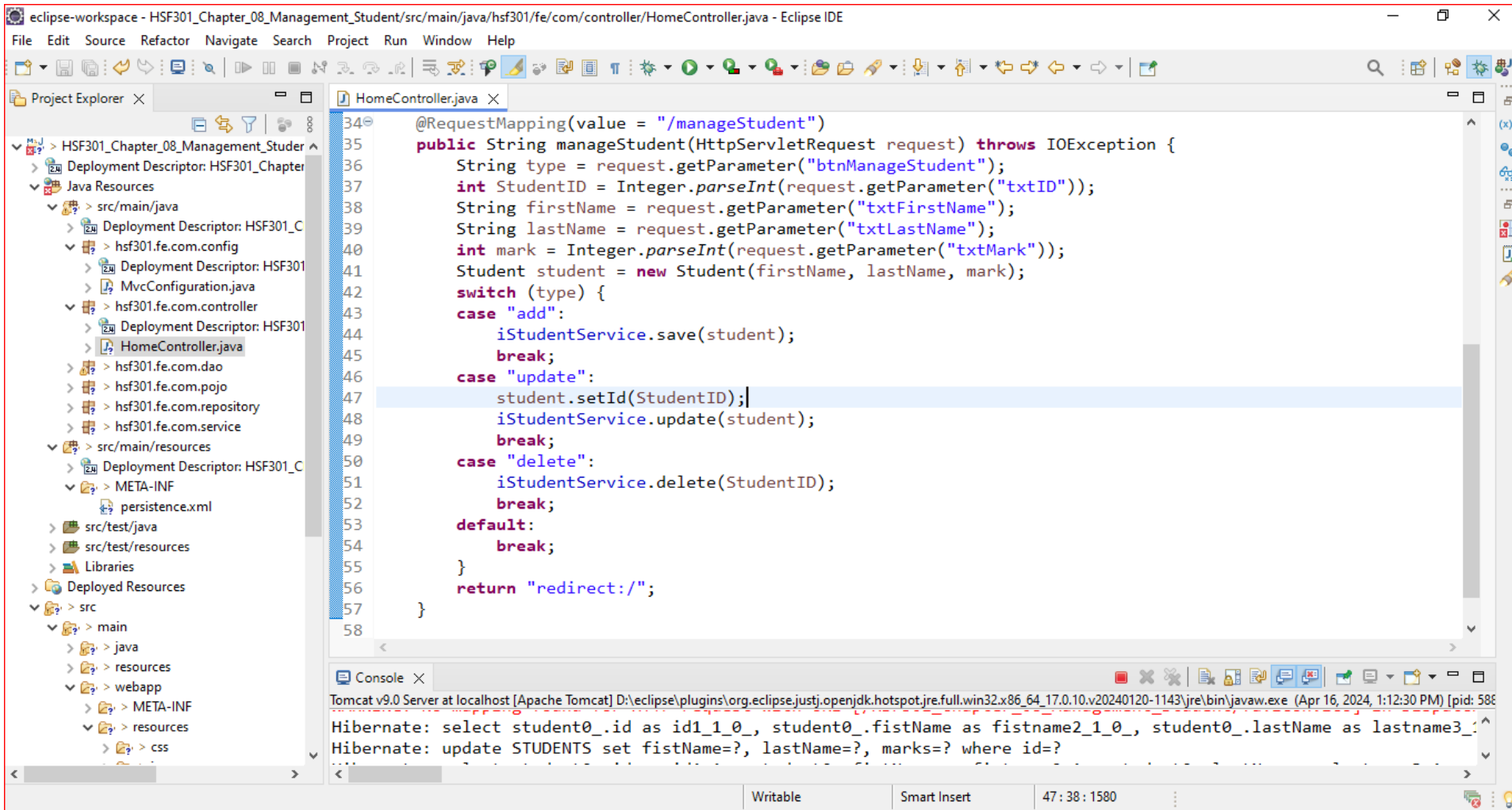
Manage Users

▼ User Lists

Choose	ID	First Name	Last Name	Mark
☐	1	Lam	Nguyen	7
☐	3	Van	Nguyen	7

Key Name	Key Value
ID	
First Name	
Last Name	
Mark	

20. Edit the HomeController.java



```
eclipse-workspace - HSF301_Chapter_08_Management_Student/src/main/java/hsf301/fe/com/controller/HomeController.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  HSF301_Chapter_08_Management_Student
    Deployment Descriptor: HSF301_Chapter
    Java Resources
      src/main/java
        Deployment Descriptor: HSF301_C
        hsf301.fe.com.config
          Deployment Descriptor: HSF301
          MvcConfiguration.java
        hsf301.fe.com.controller
          Deployment Descriptor: HSF301
          HomeController.java
        hsf301.fe.com.dao
        hsf301.fe.com.pojo
        hsf301.fe.com.repository
        hsf301.fe.com.service
      src/main/resources
        Deployment Descriptor: HSF301_C
        META-INF
          persistence.xml
      src/test/java
      src/test/resources
      Libraries
    Deployed Resources
      src
        main
          java
          resources
        webapp
          META-INF
          resources
          css

HomeController.java
34 @RequestMapping(value = "/manageStudent")
35 public String manageStudent(HttpServletRequest request) throws IOException {
36     String type = request.getParameter("btnManageStudent");
37     int StudentID = Integer.parseInt(request.getParameter("txtID"));
38     String firstName = request.getParameter("txtFirstName");
39     String lastName = request.getParameter("txtLastName");
40     int mark = Integer.parseInt(request.getParameter("txtMark"));
41     Student student = new Student(firstName, lastName, mark);
42     switch (type) {
43     case "add":
44         iStudentService.save(student);
45         break;
46     case "update":
47         student.setId(StudentID);
48         iStudentService.update(student);
49         break;
50     case "delete":
51         iStudentService.delete(StudentID);
52         break;
53     default:
54         break;
55     }
56     return "redirect:/";
57 }
58

Console
Tomcat v9.0 Server at localhost [Apache Tomcat] D:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.17.0.10.v20240120-1143\jre\bin\javaw.exe (Apr 16, 2024, 1:12:30 PM) [pid: 588]
Hibernate: select student0_.id as id1_1_0_, student0_.firstName as firstName2_1_0_, student0_.lastName as lastName3_1_0_
Hibernate: update STUDENTS set firstName=?, lastName=?, marks=? where id=?
```

21. Result

Demo Spring MVC

localhost:8080/HSF301_Chapter_08_Management_Student/

FPT DesignPattern Microbit E-Learning & Educa... Maqueen Robo t... Perplexity Consensus: AI Searc... Robot hormiga - Yo... Động cơ DC giảm t... All Bookmarks

Manage Users

▼ User Lists

Choose	ID	First Name	Last Name	Mark
☒	1	Lam	Nguyen	7
☐	3	Van	Nguyen	8

Key Name	Key Value
ID	1
First Name	Lam
Last Name	Nguyen
Mark	7

Summary

The concepts are introduced:

- ◆ Create a new Maven project in Eclipse IDE
- ◆ Add the necessary dependencies for Spring MVC and data access
- ◆ Create the Model - Define the entity classes that represent domain objects
- ◆ Define the data access object (DAO) interfaces and their implementations for interacting with the database.
- ◆ Create the Controller
- ◆ Set Up the Views
- ◆ Create the views using JSP, Thymeleaf, or another templating engine
- ◆ Implement the Business Logic
- ◆ Testing
- ◆ Run the Application